

JEGYZŐKÖNYV

Szakmai Gyakorlat II.

Backend fejlesztési szakasz

Készítette: Orosz Kristóf

Neptunkód: EYZWG9

Dátum: 2026. június 04.

Orosz Kristóf

Orosz Kristóf

Hallgató

Dédesi Péter

Szakmai konzulens

Tartalomjegyzék

Bevezetés	3
Fájlok tartalmának leírása	3
1. Feladatrészek	4
1.1 Fejlesztői környezet létrehozása, beállítása	4
1.2 Adatbázis létrehozása	8
1.3 Adatbázis feltöltése adatokkal	14
1.4 Regisztráció API	18
1.5 Bejelentkezés API	21
1.6 Ingatlanok CRUD műveletei	22
1.7 Bérlok CRUD műveletei	29
1.8 Fizetések CRUD műveletei	38
1.9 Szerződések CRUD műveletei	44
1.10 Üzenetek CRUD műveletei	51
1.11 Jogosultságkezelés és admin funkciók	54

Bevezetés

Az ingatlankezelő rendszer fejlesztésének backend szakasza során a projekt szerveroldali komponenseinek kialakítását végeztem el. A fejlesztés célja egy olyan rendszer létrehozása volt, amely lehetővé teszi az ingatlanok, bérlők, szerződések, fizetések és üzenetek kezelését egy központi adatbázis segítségével. A backend megvalósítása Node.js és Express keretrendszer használatával történt, míg az adatok tárolását MariaDB adatbázis biztosította. A fejlesztési folyamat során kialakítottam az adatbázis szerkezetét, elkészítettem a szükséges API végpontokat, valamint gondoskodtam az egyes rendszerfunkciók teszteléséről is. A fejlesztés eredményeként létrejött egy működő backend rendszer, amely képes kiszolgálni a gyakorlat későbbi szakaszában elkészülő frontend projektrészt.

Fájlok tartalmának leírása

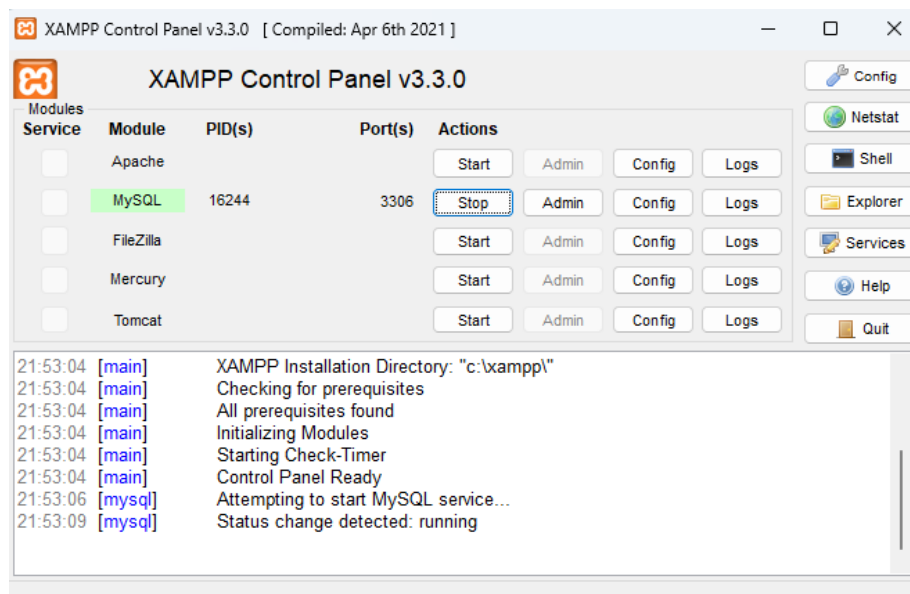
- | | |
|--|---|
| - Bevezetés.pdf | [A mappa tartalmának leírása.] |
| - Adatbázis létrehozása.pdf | [Az ingatlankezelő rendszer adatbázisának létrehozása.] |
| - Bérlők CRUD műveletei.pdf | [A bérlőkhöz kapcsolódó funkciók dokumentációja.] |
| - Fejlesztői környezet létrehozása.pdf | [A fejlesztői környezet kialakításának bemutatása.] |
| - Fizetések CRUD műveletei.pdf | [A fizetésekhez kapcsolódó CRUD funkciók.] |
| - Ingatlan CRUD műveletei.pdf | [Az ingatlanok kezelési CRUD funkciók.] |
| - Regisztrációs és bejelentkezés.pdf | [A felhasználói regisztráció és hitelesítés backend.] |
| - Szerződések CRUD műveletei.pdf | [A bérleti szerződésekhez kapcsolódó CRUD funkciók.] |
| - Üzenetek CRUD műveletei.pdf | [Az üzenetküldési funkciók dokumentációja.] |
| - JK_Tervezési szakasz.pdf | [A backend tervezési szakasz jegyzőkönyve.] |
| - Backend forrásfájlok | [A backend forrásfájlok mappája.] |
| - ingatlankezelo.sql.txt | [Az adatbázis utasítások dokumentuma.] |

1. Feladatrészek

1.1 Fejlesztői környezet létrehozása, beállítása

A fejlesztői környezet kialakítása során létrehoztam a projekt működéséhez szükséges backend és frontend struktúrát. Első lépésként elindítottam a XAMPP környezetben a MariaDB adatbázis-kezelő szolgáltatást, amely biztosította az adatbázis megfelelő működését a fejlesztés során. Ezt követően létrehoztam a backend mappát, majd inicializáltam a Node.js projektet és telepítettem a szükséges csomagokat, például az Express, CORS, MariaDB és Nodemon modulokat. A backend fejlesztése során elkészítettem a db.js fájlt, amely létrehozza a kapcsolatot a Node.js alkalmazás és a MariaDB adatbázis között. Ezután kialakítottam a server.js fájlt, amely biztosítja a backend szerver működését és az API végpontok kezelését. A frontend fejlesztéséhez létrehoztam egy React alapú projektet a create-react-app eszköz segítségével, amely automatikusan elkészítette a szükséges fájlokat és mappaszerkezetet. A React alkalmazás elindítása után ellenőriztem a frontend helyes működését a böngészőben, majd módosítottam az alapértelmezett kezdőoldalt és létrehoztam az első saját React komponenset.

A XAMPP vezérlőpult használata során elindítottam a MariaDB adatbázis-kezelő szolgáltatást a MySQL modul segítségével. A rendszer ellenőrizte a szükséges előfeltételeket és sikeresen inicializálta a szolgáltatásokat.



1. Részlet a fejlesztői környezet beállításáról.

A projekt létrehozásának első lépéseként létrehoztam a backend mappát a fejlesztői környezetben belül. A mappa kialakításával elkülönítettem a szerveroldali fájlokat és a backendhez kapcsolódó komponenseket a projekt többi részétől. Ezzel előkészítettem a Node.js és Express alapú backend fejlesztési környezet kialakítását.

```
PS C:\ingatlankezelzo> mkdir backend

Directory: C:\ingatlankezelzo

Mode                LastWriteTime         Length Name
----                -
d-----         2026. 05. 25.    21:43             backend
```

2. Részlet a fejlesztői környezet beállításáról.

A backend mappa létrehozása után beléptem a könyvtárba a terminál segítségével. Ezzel beállítottam a megfelelő munkakörnyezetet a backend fejlesztés folytatásához. A mappába történő belépéssel előkészítettem a Node.js projekt inicializálását és a szükséges csomagok telepítését.

```
PS C:\ingatlankezelolo> cd backend
```

3.Részlet a fejlesztői környezet beállításáról.

```
PS C:\ingatlankezelolo\backend> npm init -y
Wrote to C:\ingatlankezelolo\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

4.Részlet a fejlesztői környezet beállításáról.

A backend fejlesztési környezet kialakítása során telepítettem a projekt működéséhez szükséges Node.js csomagokat, például az Express, CORS, MariaDB és Nodemon modulokat. Az Express keretrendszert a backend szerver létrehozásához, a CORS modult a frontend és backend közötti kommunikáció engedélyezéséhez, míg a MariaDB csomagot az adatbázis kapcsolat kezeléséhez használtam. A Nodemon segítségével biztosítottam, hogy a szerver automatikusan újrainduljon a fájlok módosítása esetén, ezzel megkönnyítve a fejlesztési folyamatot.

```
PS C:\ingatlankezelolo\backend> npm install express cors mariadb nodemon

added 100 packages, and audited 101 packages in 7s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

5.Részlet a fejlesztői környezet beállításáról.

A React frontend projekt létrehozása során elindítottam a create-react-app telepítési folyamatát, amely automatikusan elkészítette a frontend alkalmazás alapstruktúráját. A telepítés során a rendszer létrehozta a szükséges React fájlokat, könyvtárakat és fejlesztői környezetet. Ezzel előkészítettem a modern frontend felület fejlesztését és a React alapú webalkalmazás további kialakítását.

```

PS C:\ingatlankezelzo> npx create-react-app frontend
Need to install the following packages:
create-react-app@5.1.0
Ok to proceed? (y) y

npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated uid-number@0.0.6: This package is no longer supported.
npm warn deprecated tar@2.2.2: Old versions of tar are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated fstream-ignore@1.0.5: This package is no longer supported.
npm warn deprecated fstream@1.0.12: This package is no longer supported.
create-react-app is deprecated.

You can find a list of up-to-date React frameworks on react.dev
For more info see: https://react.dev/link/cra

This error message will only be shown once per install.

Creating a new React app in C:\ingatlankezelzo\frontend.

```

6. Részlet a fejlesztői környezet beállításáról.

A React frontend projekt létrehozása során sikeresen elkészítettem a frontend alkalmazás alapstruktúráját a create-react-app eszköz segítségével. A rendszer automatikusan létrehozta a React működéséhez szükséges fájlokat, könyvtárakat és konfigurációs beállításokat.

```

PS C:\ingatlankezelzo> npx create-react-app frontend

Success! Created frontend at C:\ingatlankezelzo\frontend
Inside that directory, you can run several commands:

  npm start
    Starts the development server.

  npm run build
    Bundles the app into static files for production.

  npm test
    Starts the test runner.

  npm run eject
    Removes this tool and copies build dependencies, configuration files
    and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

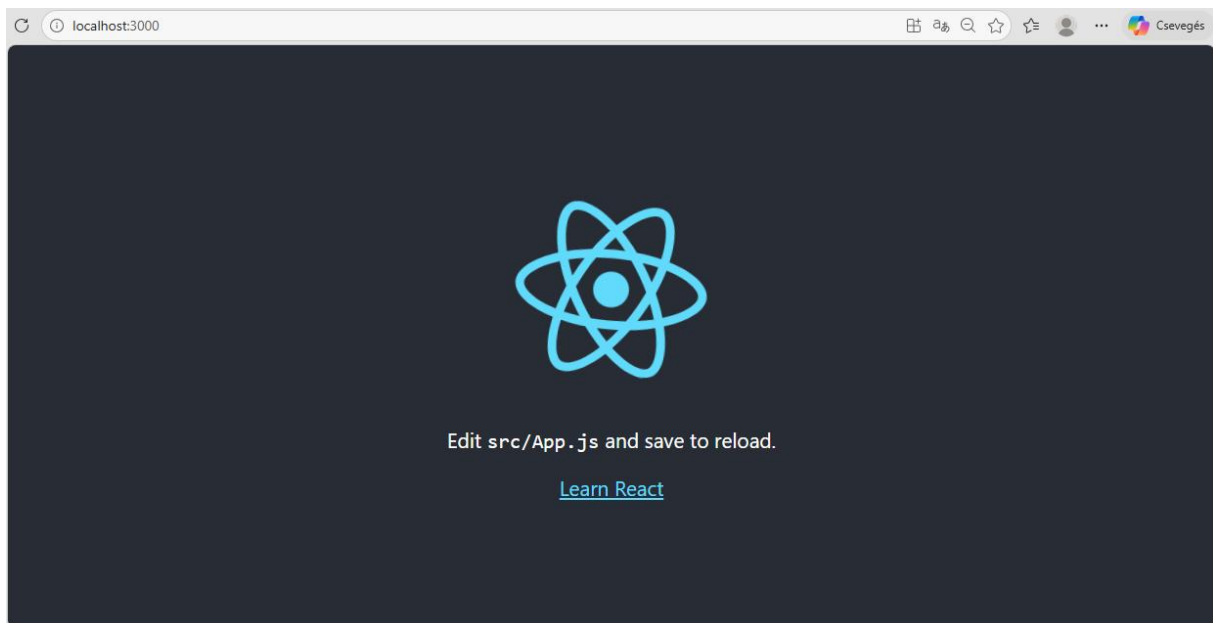
  cd frontend
  npm start

Happy hacking!

```

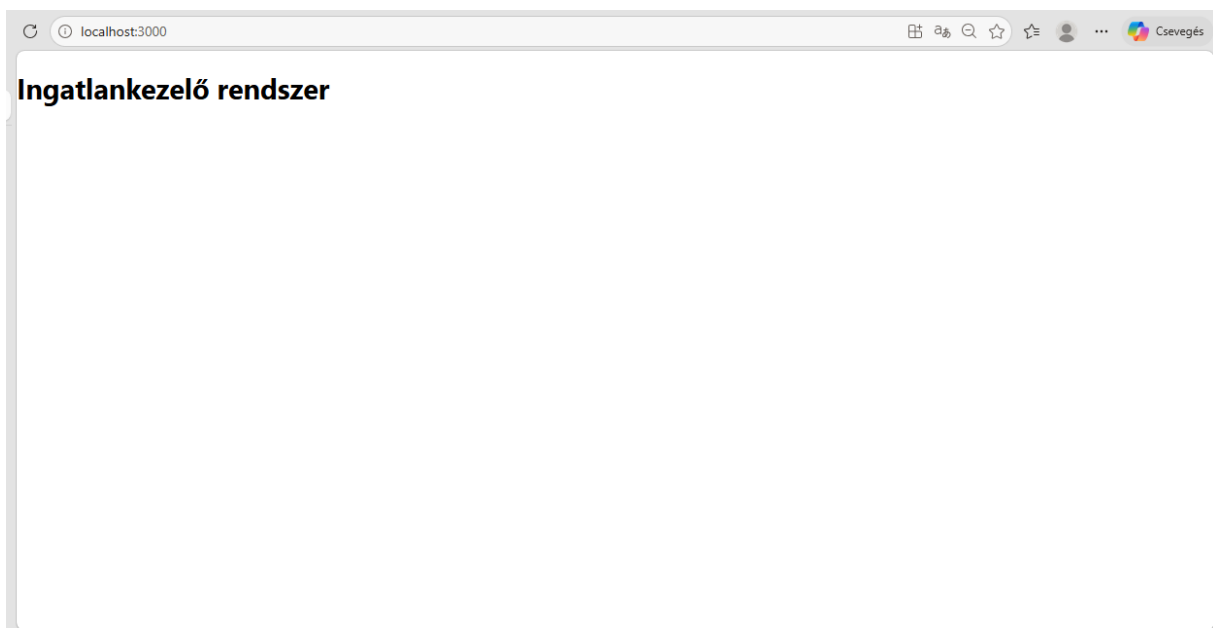
7. Részlet a fejlesztői környezet beállításáról.

A React frontend projekt elindítása után ellenőriztem, hogy a fejlesztői környezet megfelelően működik-e a böngészőben. A rendszer sikeresen megjelenítette a React alapértelmezett kezdőoldalát a localhost:3000 címen, amely igazolta a frontend alkalmazás helyes telepítését és futását.



8.Részlet a fejlesztői környezet beállításáról.

A React alkalmazás alapértelmezett kezdőoldalát módosítottam, és létrehoztam az első saját frontend oldalt. Az App.js fájl szerkesztésével megjelenítettem az „Ingatlankezelő rendszer” feliratot, amely a projekt első saját React komponenseként szolgált. Ezzel ellenőriztem, hogy a React alkalmazás megfelelően frissíti és megjeleníti a módosított felhasználói felületet a böngészőben.



9.Részlet a fejlesztői környezet beállításáról.

A db.js fájl létrehozása során kialakítottam a kapcsolatot a Node.js backend és a MariaDB adatbázis között. A konfigurációban megadtam az adatbázis eléréséhez szükséges adatokat, például a host címet, a felhasználónevet, a jelszót és az adatbázis nevét. A kapcsolat létrehozásához a MariaDB csomag createPool funkcióját használtam, amely lehetővé teszi az adatbázis-kapcsolatok hatékony kezelését a backend alkalmazásban.

```
const mariadb = require("mariadb");
```

```
const pool = mariadb.createPool({
  host: "localhost",
  user: "root",
  password: "",
  database: "ingatlankezelo",
});

module.exports = pool;
```

10.Részlet a db.js fájlból.

A server.js fájl elkészítése során kialakítottam a backend szerver alapműködését Express.js keretrendszer használatával. A fejlesztés során beállítottam a CORS támogatást és a JSON formátumú adatok kezelését, amelyek lehetővé teszik a frontend és backend közötti kommunikációt. Létrehoztam egy egyszerű API végpontot is, amely sikeres kérés esetén egy JSON válaszüzenetet küld vissza, valamint beállítottam a szerver működését a 3001-es porton.

```
const express = require("express");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {

  res.json({
    uzenet: "Backend mukodik!"
  });

});

app.listen(3001, () => {

  console.log("A backend sikeresen elindult.");

});
```

11.Részlet a server.js fájlból.

1.2 Adatbázis létrehozása

Adatbázis létrehozása

Az adatbázis létrehozása során kialakítottam az ingatlankezelő rendszer működéséhez szükséges MariaDB adatbázist és a hozzá tartozó táblákat. Elkészítettem a felhasználók, ingatlanok, bérlok, szerződések, fizetések és üzenetek adatainak tárolására szolgáló adatbázis szerkezetet. A táblák létrehozása során elsődleges és idegen kulcs kapcsolatokat alkalmaztam az adatok közötti kapcsolatok biztosítására. Az adatbázis kialakításával biztosítottam a rendszer adatainak átlátható és strukturált tárolását. A fejlesztés során SQL parancsok segítségével ellenőriztem a táblák sikeres létrehozását és megfelelő működését.

Az adatbázis létrehozása során elkészítettem az ingatlankezelő nevű MariaDB adatbázist a CREATE DATABASE SQL parancs segítségével. Ezt követően a USE utasítással kiválasztottam az adatbázist, hogy a további táblák és adatstruktúrák ebben a környezetben jöjjenek létre.

```
CREATE DATABASE ingatlankezelő;  
USE ingatlankezelő;
```

```
MariaDB [(none)]> CREATE DATABASE ingatlankezelő;  
Query OK, 1 row affected (0.003 sec)  
  
MariaDB [(none)]> USE ingatlankezelő;  
Database changed
```

1.Részlet a MariaDB -s adatbázis létrehozásáról.

Users tábla létrehozása

A users tábla létrehozása során megterveztem és elkészítettem a rendszer felhasználóinak adatainak tárolására szolgáló adatbázis táblát. A tábla kialakításakor meghatároztam a szükséges mezőket, például a felhasználó nevét, email címét, jelszavát, szerepkörét és telefonszámát, valamint beállítottam az elsődleges kulcsot és az egyedi email azonosítást is. A létrehozás után SQL lekérdezéssel ellenőriztem a tábla sikeres működését és szerkezetét a MariaDB rendszerben.

```
CREATE TABLE users (  
id INT AUTO_INCREMENT PRIMARY KEY,  
nev VARCHAR(100) NOT NULL,  
email VARCHAR(100) NOT NULL UNIQUE,  
jelszo VARCHAR(255) NOT NULL,  
szerepkor VARCHAR(50) NOT NULL,  
telefonszam VARCHAR(30),  
letrehozasdatum DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
MariaDB [ingatlankezelő]> CREATE TABLE users (  
-> id INT AUTO_INCREMENT PRIMARY KEY,  
-> nev VARCHAR(100) NOT NULL,  
-> email VARCHAR(100) NOT NULL UNIQUE,  
-> jelszo VARCHAR(255) NOT NULL,  
-> szerepkor VARCHAR(50) NOT NULL,  
-> telefonszam VARCHAR(30),  
-> letrehozasdatum DATETIME DEFAULT CURRENT_TIMESTAMP  
-> );  
Query OK, 0 rows affected (0.027 sec)  
  
MariaDB [ingatlankezelő]> SELECT*FROM users;  
Empty set (0.033 sec)
```

2.Részlet a users tábla létrehozásáról és az SQL kódjából.

Ingatlanok tábla létrehozása

Az ingatlanok tábla létrehozása során kialakítottam az ingatlanok adatainak tárolására szolgáló adatbázis szerkezetet. A tábla tartalmazza az ingatlan legfontosabb adatait, például a tulajdonos azonosítóját, az ingatlan címét, típusát, szobaszámát, havi bérleti díját és leírását. A fejlesztés során idegen kulcs

kapcsolatot is létrehoztam a users tábla és az ingatlanok tábla között, amely biztosítja az ingatlanok és tulajdonosok közötti kapcsolat megfelelő kezelését az adatbázisban.

```
CREATE TABLE ingatlanok (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  tulajdonosid INT NOT NULL,  
  cim VARCHAR(255) NOT NULL,  
  tipus VARCHAR(100),  
  szobakszama INT,  
  haviar DECIMAL(10,2),  
  leiras TEXT,  
  
  CONSTRAINT fk_ingatlan_tulajdonos  
  FOREIGN KEY (tulajdonosid)  
  REFERENCES users(id)  
  ON DELETE CASCADE  
  ON UPDATE CASCADE  
);
```

```
MariaDB [ingatlankezelo]> CREATE TABLE ingatlanok (  
->   id INT AUTO_INCREMENT PRIMARY KEY,  
->   tulajdonosid INT NOT NULL,  
->   cim VARCHAR(255) NOT NULL,  
->   tipus VARCHAR(100),  
->   szobakszama INT,  
->   haviar DECIMAL(10,2),  
->   leiras TEXT,  
->  
->   CONSTRAINT fk_ingatlan_tulajdonos  
->   FOREIGN KEY (tulajdonosid)  
->   REFERENCES users(id)  
->   ON DELETE CASCADE  
->   ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0.026 sec)  
  
MariaDB [ingatlankezelo]> SELECT * FROM ingatlanok;  
Empty set (0.001 sec)
```

3. Részlet az ingatlanok tábla létrehozásáról és az SQL kódjából.

Bérlők tábla létrehozása

A berlok tábla létrehozása során kialakítottam a rendszerben szereplő bérlők adatainak tárolására szolgáló adatbázis táblát. A tábla tartalmazza a bérlők legfontosabb adatait, például a nevüket, email címüket, telefonszámukat, személyi igazolvány számukat és lakcímüket. A fejlesztés során idegen kulcs kapcsolatot hoztam létre a users és berlok táblák között, amely biztosítja a felhasználói fiókok és a bérlői adatok összekapcsolását az adatbázisban.

```
CREATE TABLE berlok (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  userid INT NOT NULL UNIQUE,  
  nev VARCHAR(100) NOT NULL,  
  email VARCHAR(100),  
  telefonszam VARCHAR(30),  
  személyiszam VARCHAR(50),
```

```
lakcim VARCHAR(255),

CONSTRAINT fk_berlo_user
FOREIGN KEY (userid)
REFERENCES users(id)
ON DELETE CASCADE
ON UPDATE CASCADE
);
```

```
MariaDB [ingatlankezeselo]> CREATE TABLE berlok (
-> id INT AUTO_INCREMENT PRIMARY KEY,
-> userid INT NOT NULL UNIQUE,
-> nev VARCHAR(100) NOT NULL,
-> email VARCHAR(100),
-> telefonszam VARCHAR(30),
-> személyiszam VARCHAR(50),
-> lakcim VARCHAR(255),
->
-> CONSTRAINT fk_berlo_user
-> FOREIGN KEY (userid)
-> REFERENCES users(id)
-> ON DELETE CASCADE
-> ON UPDATE CASCADE
-> );
Query OK, 0 rows affected (0.024 sec)

MariaDB [ingatlankezeselo]> SELECT * FROM berlok;
Empty set (0.001 sec)
```

4.Részlet a bérlők tábla létrehozásáról és az SQL kódjából.

Szerződések tábla létrehozása

A szerzodesek tábla létrehozása során kialakítottam a bérleti szerződések adatainak tárolására szolgáló adatbázis szerkezetet. A tábla tartalmazza az ingatlan és a bérlő azonosítóját, valamint a szerződés kezdeti és végdátumát, havi díját és aktuális státuszát. A fejlesztés során idegen kulcs kapcsolatokat hoztam létre az ingatlanok és berlok táblákkal, amelyek biztosítják a szerződések megfelelő összekapcsolását az ingatlanokkal és a bérlőkkel az adatbázisban.

```
CREATE TABLE szerzodesek (
id INT AUTO_INCREMENT PRIMARY KEY,
ingatlanid INT NOT NULL,
berloid INT NOT NULL,
kezdetdatum DATE NOT NULL,
vegdatum DATE NOT NULL,
havidij DECIMAL(10,2),
statusz VARCHAR(50),

CONSTRAINT fk_szerzodes_ingatlan
FOREIGN KEY (ingatlanid)
REFERENCES ingatlanok(id)
ON DELETE CASCADE
ON UPDATE CASCADE,

CONSTRAINT fk_szerzodes_berlo
FOREIGN KEY (berloid)
```

```
REFERENCES berlok(id)
ON DELETE CASCADE
ON UPDATE CASCADE
);
```

```
MariaDB [ingatlankezeselo]> CREATE TABLE szerzodesek (
-> id INT AUTO_INCREMENT PRIMARY KEY,
-> ingatlanid INT NOT NULL,
-> berloid INT NOT NULL,
-> kezdetdatum DATE NOT NULL,
-> vegdatum DATE NOT NULL,
-> havidij DECIMAL(10,2),
-> statusz VARCHAR(50),
->
-> CONSTRAINT fk_szerzodes_ingatlan
-> FOREIGN KEY (ingatlanid)
-> REFERENCES ingatlanok(id)
-> ON DELETE CASCADE
-> ON UPDATE CASCADE,
->
-> CONSTRAINT fk_szerzodes_berlo
-> FOREIGN KEY (berloid)
-> REFERENCES berlok(id)
-> ON DELETE CASCADE
-> ON UPDATE CASCADE
-> );
Query OK, 0 rows affected (0.027 sec)

MariaDB [ingatlankezeselo]>
MariaDB [ingatlankezeselo]> SELECT * FROM szerzodesek;
Empty set (0.001 sec)
```

5.Részlet a szerzodesek tábla létrehozásáról és az SQL kódjából.

Fizetések tábla létrehozása

A fizetesek tábla létrehozása során kialakítottam a bérleti szerződésekhez tartozó befizetések adatainak tárolására szolgáló adatbázis szerkezetet. A tábla tartalmazza a szerződés azonosítóját, a befizetett összeget, a fizetés dátumát, a fizetés státuszát, valamint a fizetési módot és a megjegyzéseket is. A fejlesztés során idegen kulcs kapcsolatot hoztam létre a szerzodesek és fizetesek táblák között, amely biztosítja a fizetések és a hozzájuk tartozó szerződések megfelelő összekapcsolását az adatbázisban.

```
CREATE TABLE fizetesek (
id INT AUTO_INCREMENT PRIMARY KEY,
szerzodesid INT NOT NULL,
osszeg DECIMAL(10,2) NOT NULL,
fizetesdatum DATE NOT NULL,
statusz VARCHAR(50),
fizetesmod VARCHAR(50),
megjegyzes TEXT,

CONSTRAINT fk_fizetes_szerzodes
FOREIGN KEY (szerzodesid)
REFERENCES szerzodesek(id)
ON DELETE CASCADE
ON UPDATE CASCADE
);
```

```

MariaDB [ingatlankezo]> CREATE TABLE fizetesek (
->   id INT AUTO_INCREMENT PRIMARY KEY,
->   szerzodesid INT NOT NULL,
->   osszeg DECIMAL(10,2) NOT NULL,
->   fizetesdatum DATE NOT NULL,
->   statusz VARCHAR(50),
->   fizetesmod VARCHAR(50),
->   megjegyzes TEXT,
->
->   CONSTRAINT fk_fizetes_szerzodes
->   FOREIGN KEY (szerzodesid)
->   REFERENCES szerzodesek(id)
->   ON DELETE CASCADE
->   ON UPDATE CASCADE
-> );
Query OK, 0 rows affected (0.021 sec)

MariaDB [ingatlankezo]> ;
ERROR: No query specified

MariaDB [ingatlankezo]> SELECT * FROM fizetesek;
Empty set (0.001 sec)

```

6.Részlet az fizetesek tábla létrehozásáról és az SQL kódjából.

Üzenetek tábla létrehozása

Az üzenetek tábla létrehozása során kialakítottam a felhasználók közötti üzenetek tárolására szolgáló adatbázis szerkezetet. A tábla tartalmazza az üzenet küldőjének azonosítóját, az üzenet tárgyát, szövegét és a küldés időpontját is. A fejlesztés során idegen kulcs kapcsolatot hoztam létre a berlok és üzenetek táblák között, amely biztosítja az üzenetek és a hozzájuk tartozó felhasználók megfelelő összekapcsolását az adatbázisban.

```

CREATE TABLE uzenetek (
id INT AUTO_INCREMENT PRIMARY KEY,
kuldoid INT NOT NULL,
targy VARCHAR(255),
uzenet TEXT NOT NULL,
kuldesdatum DATETIME DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT fk_uzenet_kuldo
FOREIGN KEY (kuldoid)
REFERENCES berlok(id)
ON DELETE CASCADE
ON UPDATE CASCADE
);

```

```

MariaDB [ingatlankezelő]> CREATE TABLE uzenetek (
->     id INT AUTO_INCREMENT PRIMARY KEY,
->     kuldoid INT NOT NULL,
->     targy VARCHAR(255),
->     uzenet TEXT NOT NULL,
->     kuldesdatum DATETIME DEFAULT CURRENT_TIMESTAMP,
->
->     CONSTRAINT fk_uzenet_kuldo
->     FOREIGN KEY (kuldoid)
->     REFERENCES berlok(id)
->     ON DELETE CASCADE
->     ON UPDATE CASCADE
-> );
Query OK, 0 rows affected (0.022 sec)

MariaDB [ingatlankezelő]> SELECT * FROM uzenetek;
Empty set (0.001 sec)

```

7.Részlet az uzenetek tábla létrehozásáról és az SQL kódjából.

1.3. Adatbázis táblák feltöltése

Az adatbázis feltöltési szakasz során elkészítettem a rendszer működéséhez szükséges tesztadatokat. A munka során feltöltöttem a users, berlok, ingatlanok, szerzodesek, fizetesek és uzenetek táblákat valóság-hű adatokkal SQL INSERT utasítások segítségével. Külön figyelmet fordítottam az idegen kulcs kapcsolatok helyes kezelésére, hogy az egyes rekordok megfelelően kapcsolódjanak egymáshoz. A feltöltés során adminisztrátor és bérlő felhasználókat, ingatlanokat, bérleti szerződéseket, fizetési adatokat és mintául szolgáló üzeneteket rögzítettem az adatbázisban. Az adatok beszúrását követően SQL lekérdezésekkel ellenőriztem a rekordok sikeres tárolását és a kapcsolatok helyes működését.

Users tábla feltöltése

A users tábla feltöltése során létrehoztam a rendszer alapfelhasználóit, amelyek között adminisztrátorok és bérlők is szerepelnek. Az adatok rögzítése SQL INSERT utasítás segítségével történt, amely egyszerre több rekord beszúrását tette lehetővé. A felhasználókhöz nevet, e-mail címet, jelszót, szerepkört és telefonszámot adtam meg. A feltöltés után SQL lekérdezéssel ellenőriztem, hogy az adatok megfelelően bekerültek-e az adatbázisba. Az ellenőrzés során meggyőződtem arról, hogy a rendszerben tárolt felhasználók adatai helyesen jelennek meg.

```

INSERT INTO users (nev, email, jelszo, szerepkor, telefonszam) VALUES
('Kovács István', 'istvan.kovacs@gmail.com', '1234', 'admin', '06301111111'),
('Nagy József', 'jozsef.nagy@gmail.com', '1234', 'admin', '06302222222'),

('Tóth Péter', 'peter.toth@gmail.com', '1234', 'berlo', '06303333333'),
('Kiss Anna', 'anna.kiss@gmail.com', '1234', 'berlo', '06304444444'),
('Szabó Dávid', 'david.szabo@gmail.com', '1234', 'berlo', '06305555555'),
('Varga Éva', 'eva.varga@gmail.com', '1234', 'berlo', '06306666666'),
('Molnár Gábor', 'gabor.molnar@gmail.com', '1234', 'berlo', '06307777777'),
('Farkas Dóra', 'dora.farkas@gmail.com', '1234', 'berlo', '06308888888'),
('Horváth Balázs', 'balazs.horvath@gmail.com', '1234', 'berlo', '06309999999'),
('Lakatos Krisztián', 'krisztian.lakatos@gmail.com', '1234', 'berlo', '06301234567'),
('Papp Renáta', 'renata.papp@gmail.com', '1234', 'berlo', '06307654321'),
('Bíró Zoltán', 'zoltan.biro@gmail.com', '1234', 'berlo', '06302345678');

```

```
MariaDB [ingatlankezeles]> SELECT * FROM users;
```

id	nev	email	jelszo	szerep	telefonszam	lethoz
1	Teszt Bela	teszt@teszt.hu	123456	berlo	06301234567	2026-05-25 22:35:35
2	Kovács István	istvan.kovacs@gmail.com	1234	admin	06301111111	2026-06-02 22:30:14
3	Nagy József	jozsef.nagy@gmail.com	1234	admin	06302222222	2026-06-02 22:30:14
4	Tóth Péter	peter.toth@gmail.com	1234	berlo	06303333333	2026-06-02 22:30:14
5	Kiss Anna	anna.kiss@gmail.com	1234	berlo	06304444444	2026-06-02 22:30:14
6	Szabó Dávid	david.szabo@gmail.com	1234	berlo	06305555555	2026-06-02 22:30:14
7	Varga Éva	eva.varga@gmail.com	1234	berlo	06306666666	2026-06-02 22:30:14
8	Molnár Gábor	gabor.molnar@gmail.com	1234	berlo	06307777777	2026-06-02 22:30:14
9	Farkas Dóra	dora.farkas@gmail.com	1234	berlo	06308888888	2026-06-02 22:30:14
10	Horváth Balázs	balazs.horvath@gmail.com	1234	berlo	06309999999	2026-06-02 22:30:14
11	Lakatos Krisztián	krisztian.lakatos@gmail.com	1234	berlo	06301234567	2026-06-02 22:30:14
12	Papp Renáta	renata.papp@gmail.com	1234	berlo	06307654321	2026-06-02 22:30:14
13	Bíró Zoltán	zoltan.biro@gmail.com	1234	berlo	06302345678	2026-06-02 22:30:14

1.kép: A users adatbázis feltöltéséről.

Bérlők tábla feltöltése

A bérlők tábla feltöltése során létrehoztam a rendszerben szereplő bérlők adatait. Minden bérlőhöz hozzárendeltem egy felhasználói azonosítót, valamint megadtam a nevét, e-mail címét, telefonszámát, személyi igazolvány számát és lakcímét. A feltöltés során figyelembe vettem a users és berlok táblák közötti kapcsolatot is. Az adatok rögzítése SQL INSERT utasítással történt. A művelet befejezése után lekérdezéssel ellenőriztem a sikeres adatfelvitelt.

```
INSERT INTO berlok (userid, nev, email, telefonszam, személyiszam, lakcim) VALUES
(3,'Tóth Péter','peter.toth@gmail.com','06303333333','AA123456','Budapest'),
(4,'Kiss Anna','anna.kiss@gmail.com','06304444444','BB234567','Debrecen'),
(5,'Szabó Dávid','david.szabo@gmail.com','06305555555','CC345678','Miskolc'),
(6,'Varga Éva','eva.varga@gmail.com','06306666666','DD456789','Nyíregyháza'),
(7,'Molnár Gábor','gabor.molnar@gmail.com','06307777777','EE567890','Eger'),
(8,'Farkas Dóra','dora.farkas@gmail.com','06308888888','FF678901','Szeged'),
(9,'Horváth Balázs','balazs.horvath@gmail.com','06309999999','GG789012','Pécs'),
(10,'Lakatos Krisztián','krisztian.lakatos@gmail.com','06301234567','HH890123','Győr'),
(11,'Papp Renáta','renata.papp@gmail.com','06307654321','II901234','Kecskemét'),
(12,'Bíró Zoltán','zoltan.biro@gmail.com','06302345678','JJ012345','Székesfehérvár');
```

```
MariaDB [ingatlankezeles]> SELECT * FROM berlok;
```

id	userid	nev	email	telefonszam	szemelyiszam	lakci
2	1	Kiss Bela	kissbela@gmail.com	06301234567	AA123456	Budapest
3	3	Tóth Péter	peter.toth@gmail.com	06303333333	AA123456	Budapest
4	4	Kiss Anna	anna.kiss@gmail.com	06304444444	BB234567	Debrecen
5	5	Szabó Dávid	david.szabo@gmail.com	06305555555	CC345678	Miskolc
6	6	Varga Éva	eva.varga@gmail.com	06306666666	DD456789	Nyíregyháza
7	7	Molnár Gábor	gabor.molnar@gmail.com	06307777777	EE567890	Eger
8	8	Farkas Dóra	dora.farkas@gmail.com	06308888888	FF678901	Szeged
9	9	Horváth Balázs	balazs.horvath@gmail.com	06309999999	GG789012	Pécs
10	10	Lakatos Krisztián	krisztian.lakatos@gmail.com	06301234567	HH890123	Győr
11	11	Papp Renáta	renata.papp@gmail.com	06307654321	II901234	Kecskemét
12	12	Bíró Zoltán	zoltan.biro@gmail.com	06302345678	JJ012345	Székesfehérvár

2.kép: A berlok adatbázis feltöltéséről.

Ingyatlanok tábla feltöltése

Az ingatlanok tábla feltöltése során rögzítettem a rendszerben nyilvántartott ingatlanok adatait. Az egyes rekordokhoz megadtam a tulajdonos azonosítóját, az ingatlan címét, típusát, szobaszámát, havi bérleti díját és rövid leírását. A fejlesztés során figyeltem arra, hogy minden ingatlan egy létező tulajdonoshoz kapcsolódjon. Az adatok beszúrását SQL utasításokkal végeztem el. A feltöltés után lekérdezéssel ellenőriztem az adatok helyes tárolását.

```
INSERT INTO ingatlanok (tulajdonosid, cim, tipus, szobakszama, haviar, leiras) VALUES
(1,'Budapest, Kossuth utca 12.','Panel',2,150000,'Felújított panel lakás'),
(1,'Budapest, Petőfi utca 8.','Társasház',3,180000,'Belvárosi lakás'),
(1,'Debrecen, Fő utca 21.','Ház',4,220000,'Családi ház'),
(1,'Nyíregyháza, Sóstó utca 4.','Ház',5,250000,'Kertes ingatlan'),
(1,'Miskolc, Arany János utca 7.','Panel',2,140000,'Jó állapotú lakás'),
(2,'Szeged, Tisza utca 15.','Garzon',1,120000,'Belvárosi garzon'),
(2,'Pécs, Rákóczi utca 3.','Társasház',2,160000,'Felújított lakás'),
(2,'Győr, Bartók Béla utca 10.','Ház',4,210000,'Nagy családi ház'),
(2,'Kecskemét, Dózsa György út 5.','Panel',2,145000,'Központi elhelyezkedés'),
(2,'Eger, Vár utca 11.','Társasház',3,170000,'Újszerű ingatlan');
```

```
MariaDB [ingatlankezezo]>
MariaDB [ingatlankezezo]> SELECT * FROM ingatlanok;
```

id	tulajdonosid	cim	tipus	szobakszama	haviar	leiras
2	1	Budapest, Kossuth utca 10.	Tarsashaz	3	180000.00	Felujitott lakas
3	1	Budapest, Kossuth utca 10.	Tarsashaz	3	180000.00	Felujitott lakas
4	1	Budapest, Kossuth utca 12.	Panel	2	150000.00	Felujitott panel lakas
5	1	Budapest, Petofi utca 8.	Tarsashaz	3	180000.00	Belvarosi lakas
6	1	Debrecen, Fő utca 21.	Ház	4	220000.00	Családi ház
7	1	Nyiregyháza, Sóstó utca 4.	Ház	5	250000.00	Kertes ingatlan
8	1	Miskolc, Arany János utca 7.	Panel	2	140000.00	Jó állapotú lakás
9	2	Szeged, Tisza utca 15.	Garzon	1	120000.00	Belvárosi garzon
10	2	Pécs, Rákóczi utca 3.	Társasház	2	160000.00	Felújított lakás
11	2	Győr, Bartók Béla utca 10.	Ház	4	210000.00	Nagy családi ház
12	2	Kecskemét, Dózsa György út 5.	Panel	2	145000.00	Központi elhelyezkedés
13	2	Eger, Vár utca 11.	Társasház	3	170000.00	Újszerű ingatlan

12 rows in set (0.000 sec)

3.kép: Az ingatlanok adatbázis feltöltéséről.

Szerződések tábla feltöltése

A szerződések tábla feltöltése során létrehoztam az ingatlanokhoz és bérlőkhöz kapcsolódó bérleti szerződéseket. Minden szerződéshez megadtam az ingatlan és a bérlő azonosítóját, valamint a szerződés kezdeti és végdátumát, havi díját és státuszát. A fejlesztés során figyelembe vettem az idegen kulcs kapcsolatokat, hogy csak létező rekordokra hivatkozzanak a szerződések. Az adatok rögzítése SQL INSERT utasításokkal történt. A feltöltés sikerességét SQL lekérdezéssel ellenőriztem.

```
INSERT INTO szerzodesek
(ingatlanid, berloid, kezdetdatum, vegdatum, havidij, statusz)
VALUES
(2,2,'2026-01-01','2026-12-31',150000,'Aktív'),
(3,3,'2026-01-01','2026-12-31',180000,'Aktív'),
(4,4,'2026-02-01','2027-01-31',220000,'Aktív'),
(5,5,'2026-03-01','2027-02-28',250000,'Aktív'),
(6,6,'2026-01-15','2026-12-15',140000,'Aktív'),
(7,7,'2026-04-01','2027-03-31',120000,'Aktív'),
```

```
(8,8,'2026-05-01','2027-04-30',160000,'Aktív'),
(9,9,'2026-06-01','2027-05-31',210000,'Aktív'),
(10,10,'2026-07-01','2027-06-30',145000,'Aktív'),
(11,11,'2026-08-01','2027-07-31',170000,'Aktív');
```

```
MariaDB [ingatlankezelelo]> SELECT * FROM szerzodesek;
```

id	ingatlanid	berloid	kezdetdatum	vegdatum	havidi	statusz
4	3	2	2026-01-01	2026-12-31	180000.00	Aktív
5	3	2	2026-01-01	2026-12-31	180000.00	Aktív
16	2	2	2026-01-01	2026-12-31	150000.00	Aktív
17	3	3	2026-01-01	2026-12-31	180000.00	Aktív
18	4	4	2026-02-01	2027-01-31	220000.00	Aktív
19	5	5	2026-03-01	2027-02-28	250000.00	Aktív
20	6	6	2026-01-15	2026-12-15	140000.00	Aktív
21	7	7	2026-04-01	2027-03-31	120000.00	Aktív
22	8	8	2026-05-01	2027-04-30	160000.00	Aktív
23	9	9	2026-06-01	2027-05-31	210000.00	Aktív
24	10	10	2026-07-01	2027-06-30	145000.00	Aktív
25	11	11	2026-08-01	2027-07-31	170000.00	Aktív

```
12 rows in set (0.000 sec)
```

4.kép: A szerződések adatbázis feltöltéséről.

Fizetések tábla feltöltése

A fizetések tábla feltöltése során rögzítettem a bérleti szerződésekhez kapcsolódó befizetéseket. Az adatok között szerepelt a szerződés azonosítója, a befizetett összeg, a fizetés dátuma, a státusz, a fizetési mód és egy rövid megjegyzés. A fejlesztés során ügyeltem arra, hogy minden fizetés egy létező szerződéshez kapcsolódjon. Az adatok feltöltését SQL INSERT utasítások segítségével valósítottam meg. A művelet után ellenőriztem, hogy minden rekord megfelelően bekerült az adatbázisba.

```
INSERT INTO fizetesek
(szerzodesid, osszeg, fizetesdatum, statusz, fizetesmod, megjegyzes)
VALUES
(4,150000,'2026-01-05','Befizetve','Átutalás','Januári bérleti díj'),
(5,180000,'2026-01-05','Befizetve','Bankkártya','Januári bérleti díj'),
(16,220000,'2026-02-05','Befizetve','Átutalás','Februári bérleti díj'),
(17,250000,'2026-03-05','Befizetve','Átutalás','Márciusi bérleti díj'),
(18,140000,'2026-01-20','Befizetve','Készpénz','Januári bérleti díj'),
(19,120000,'2026-04-05','Befizetve','Bankkártya','Áprilisi bérleti díj'),
(20,160000,'2026-05-05','Befizetve','Átutalás','Májusi bérleti díj'),
(21,210000,'2026-06-05','Befizetve','Átutalás','Júniusi bérleti díj'),
(22,145000,'2026-07-05','Befizetve','Készpénz','Júliusi bérleti díj'),
(23,170000,'2026-08-05','Befizetve','Bankkártya','Augusztusi bérleti díj');
```

```
MariaDB [ingatlankezo]> SELECT * FROM fizetesek;
```

id	szerzodesid	osszeg	fizetesdatum	statusz	fizetesmod	megjegyzes
2	4	180000.00	2026-01-10	Befizetve	Atutalas	Januari lakber
23	4	150000.00	2026-01-05	Befizetve	Atutalas	Januari bérleti díj
24	5	180000.00	2026-01-05	Befizetve	Bankkartya	Januari bérleti díj
25	16	220000.00	2026-02-05	Befizetve	Atutalas	Februári bérleti díj
26	17	250000.00	2026-03-05	Befizetve	Atutalas	Márciusi bérleti díj
27	18	140000.00	2026-01-20	Befizetve	Készpénz	Januari bérleti díj
28	19	120000.00	2026-04-05	Befizetve	Bankkartya	Áprilisi bérleti díj
29	20	160000.00	2026-05-05	Befizetve	Atutalas	Májusi bérleti díj
30	21	210000.00	2026-06-05	Befizetve	Atutalas	Júniusi bérleti díj
31	22	145000.00	2026-07-05	Befizetve	Készpénz	Júliusi bérleti díj
32	23	170000.00	2026-08-05	Befizetve	Bankkartya	Augusztusi bérleti díj

```
11 rows in set (0.000 sec)
```

5.kép: A fizetések adatbázis feltöltéséről.

Üzenetek tábla feltöltése

Az üzenetek tábla feltöltése során létrehoztam a rendszerben tárolt mintául szolgáló felhasználói üzeneteket. Minden rekordhoz megadtam a küldő azonosítóját, az üzenet tárgyát és az üzenet szövegét. Az adatok kialakításakor valós helyzeteket modelleztem, például hibabejelentéseket és információkéréseket. Az adatok rögzítése SQL INSERT utasításokkal történt. A feltöltés után SQL lekérdezéssel ellenőriztem az üzenetek sikeres tárolását az adatbázisban.

```
INSERT INTO uzenetek
```

```
(kulldoid, targy, uzenet)
```

```
VALUES
```

```
(2,'Fűtési probléma','A nappaliban nem működik megfelelően a fűtés.'),
```

```
(3,'Csőtörés bejelentése','A konyhában szivárog a víz a mosogató alatt.'),
```

```
(4,'Kulcsmásolat igénylése','Szeretnék egy pótkulcsot kérni az ingatlanhoz.'),
```

```
(5,'Villanykapcsoló hiba','A hálósobában nem működik a villanykapcsoló.'),
```

```
(6,'Festési igény','A lakás egyik falának újrafestését szeretném kérni.'),
```

```
(7,'Internet probléma','Nem működik az internetkapcsolat az ingatlanban.'),
```

```
(8,'Parkolóhely kérés','Érdeklődnék a parkolóhely bérletének lehetőségéről.'),
```

```
(9,'Lift meghibásodás','A társasház liftje nem üzemel megfelelően.'),
```

```
(10,'Ablak javítás','Az egyik ablak nehezen zárható és huzatos.'),
```

```
(11,'Szerződés hosszabbítás','Szeretnék érdeklődni a szerződés meghosszabbításáról.');
```

```
MariaDB [ingatlankezo]> SELECT * FROM uzenetek;
```

id	kulldoid	targy	uzenet	kuldesdatum
1	2	Hibabejelentés	A futes nem mukodik a lakasban.	2026-06-02 22:13:21
12	2	Fűtési probléma	A nappaliban nem működik megfelelően a fűtés.	2026-06-02 22:48:24
13	3	Csőtörés bejelentése	A konyhában szivárog a víz a mosogató alatt.	2026-06-02 22:48:24
14	4	Kulcsmásolat igénylése	Szeretnék egy pótkulcsot kérni az ingatlanhoz.	2026-06-02 22:48:24
15	5	Villanykapcsoló hiba	A hálósobában nem működik a villanykapcsoló.	2026-06-02 22:48:24
16	6	Festési igény	A lakás egyik falának újrafestését szeretném kérni.	2026-06-02 22:48:24
17	7	Internet probléma	Nem működik az internetkapcsolat az ingatlanban.	2026-06-02 22:48:24
18	8	Parkolóhely kérés	Érdeklődnék a parkolóhely bérletének lehetőségéről.	2026-06-02 22:48:24
19	9	Lift meghibásodás	A társasház liftje nem üzemel megfelelően.	2026-06-02 22:48:24
20	10	Ablak javítás	Az egyik ablak nehezen zárható és huzatos.	2026-06-02 22:48:24
21	11	Szerződés hosszabbítás	Szeretnék érdeklődni a szerződés meghosszabbításáról.	2026-06-02 22:48:24

```
11 rows in set (0.000 sec)
```

6.kép: Az üzenetek adatbázis feltöltéséről.

1.4 Regisztráció API

A regisztrációs backend végpont elkészítése során kialakítottam a felhasználók adatainak rögzítésére szolgáló API működését. A fejlesztés során a rendszer fogadja a frontendről érkező adatokat, majd ellenőrzi, hogy az adott email cím már szerepel-e az adatbázisban. Amennyiben az email cím még nem

létezik, a rendszer eltárolja az új felhasználó adatait a users táblában, majd sikeres regisztrációs visszajelzést küld a kliens oldal számára.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    nev,
    email,
    jelszo,
    szerepkor,
    telefonszam
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    const letezik = await conn.query(
      "SELECT * FROM users WHERE email = ?",
      [email]
    );

    if (letezik.length > 0) {

      return res.json({
        siker: false,
        uzenet: "Ez az email már letezik!"
      });

    }

    await conn.query(
      `INSERT INTO users
      (nev, email, jelszo, szerepkor, telefonszam)
      VALUES (?, ?, ?, ?, ?)`,
      [
        nev,
        email,
        jelszo,
        szerepkor,
        telefonszam
      ]
    );

  }
```

```

    res.json({
      siker: true,
      uzenet: "Sikeres regisztracio!"
    });

  } catch (err) {

    res.status(500).json({
      siker: false,
      hiba: err
    });

  } finally {

    if (conn) conn.release();

  }

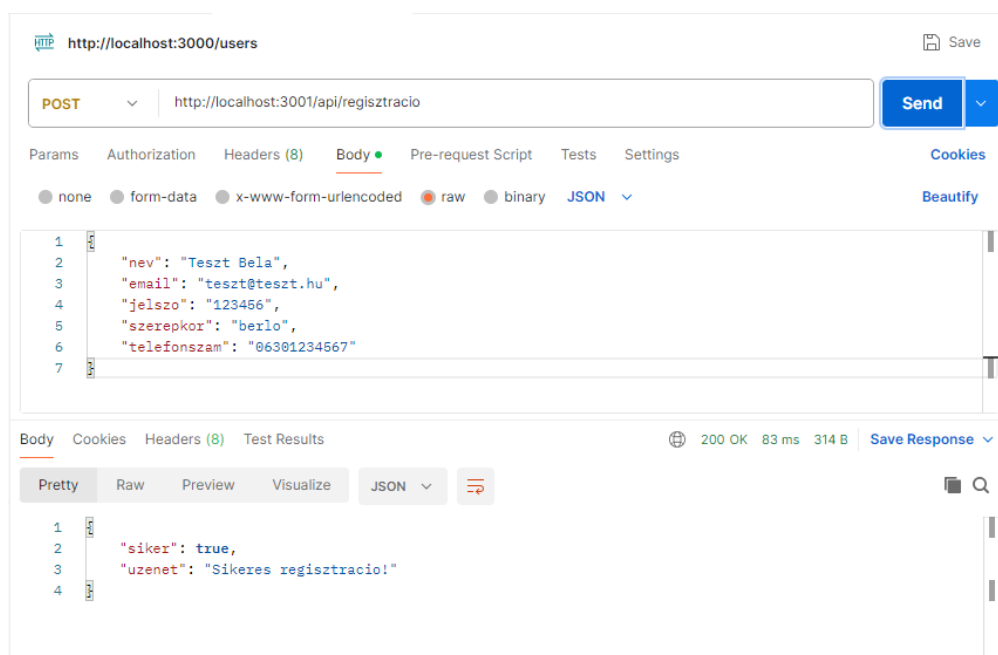
});

module.exports = router;

```

1.Részlet a regisztracio.js fájlból.

A regisztráció API tesztelése során Postman alkalmazás segítségével ellenőriztem a backend működését és az adatbázis kapcsolat helyességét. A teszt során JSON formátumban küldtem el a felhasználó adatait a regisztrációs végpontra, amelyet a rendszer sikeresen feldolgozott és eltárolt a MariaDB adatbázisban.



2.Részlet a regisztracio.js fájl Postman teszteléséből.

1.5 Bejelentkezés API

A bejelentkezés API elkészítése során kialakítottam a felhasználók hitelesítéséhez szükséges backend végpontot. A fejlesztés során a rendszer ellenőrzi az adatbázisban tárolt email címet és jelszót, majd sikeres egyezés esetén visszaküldi a felhasználó legfontosabb adatait és a sikeres bejelentkezés állapotát. A működés tesztelésével ellenőriztem, hogy a backend megfelelően kezeli a felhasználói belépést és a MariaDB adatbázissal való kommunikációt.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    email,
    jelszo
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    const users = await conn.query(
      "SELECT * FROM users WHERE email = ? AND jelszo = ?",
      [email, jelszo]
    );

    if (users.length === 0) {

      return res.json({
        siker: false,
        uzenet: "Hibas email vagy jelszo!"
      });

    }

    const user = users[0];

    res.json({
      siker: true,
      uzenet: "Sikeres bejelentkezés!",
      user: {
        id: user.id,
        nev: user.nev,
        email: user.email,
        szerepkor: user.szerepkor
      }
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({
      siker: false,
      uzenet: "Szerverhiba!"
    });
  }
});
```

```

    }
  });

  } catch (err) {

    res.status(500).json({
      siker: false,
      hiba: err
    });

  } finally {

    if (conn) conn.release();

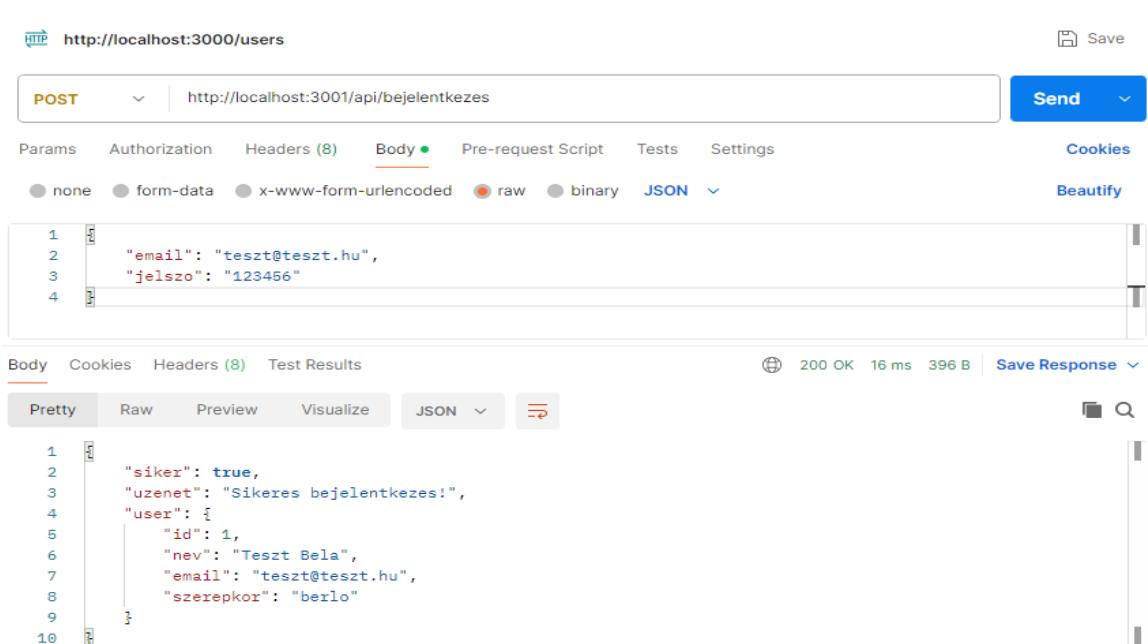
  }

});

module.exports = router;

```

1. Részlet a bejelentkezés.js fájlból.



2. Részlet a bejelentkezés.js fájl Postman teszteléséből.

1.6. Ingyatlan CRUD műveletei

A munkafolyamat során elkészítettem az Ingyatlankezelő rendszer ingatlanok kezeléséhez kapcsolódó CRUD funkcióit. Fejlesztettem az ingatlanok létrehozására, módosítására, törlésére és listázására szolgáló backend API végpontokat Node.js és Express keretrendszer használatával. A fejlesztés során kialakítottam az adatbázis és a backend közötti kapcsolatot, valamint megvalósítottam az adatok fogadását és kezelését MariaDB adatbázisban. Külön

figyelmet fordítottam a hibakezelésre és az adatbázis-kapcsolatok megfelelő lezárására. Az elkészült végpontokat Postman alkalmazás segítségével teszteltem, ahol ellenőriztem a sikeres adatfelvitelt, módosítást, törlést és lekérdezést.

Ingyatlan létrehozás:

Az ingatlan létrehozása funkció fejlesztése során elkészítettem az új ingatlanok rögzítésére szolgáló backend API végpontot. A megoldás lehetővé teszi az ingatlan legfontosabb adatainak, például címének, típusának, szobaszámának és havi bérleti díjának eltárolását az adatbázisban. A fejlesztés során kialakítottam az adatbázis-kapcsolatot és az adatok fogadását a kliens oldaltól. A végpont sikeres adatmentés esetén visszajelzést küld a felhasználó számára. Az elkészült funkció működését Postman alkalmazás segítségével teszteltem és ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    tulajdonosid,
    cim,
    tipus,
    szobakszama,
    haviar,
    leiras
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `INSERT INTO ingatlanok
      (tulajdonosid, cim, tipus, szobakszama, haviar, leiras)
      VALUES (?, ?, ?, ?, ?, ?)` ,
      [
        tulajdonosid,
        cim,
        tipus,
        szobakszama,
        haviar,
        leiras
      ]
    );

  }
```

```

        res.json({
            siker: true,
            uzenet: "Ingatlan sikeresen létrehozva!"
        });

    } catch (err) {

        res.status(500).json(err);

    } finally {

        if(conn) conn.release();

    }

});

module.exports = router;

```

The screenshot shows a Postman interface for a POST request to `http://localhost:3001/api/ingatlanletrehozaz`. The request body is a JSON object with the following properties:

```

{
  "tulajdonosid": 1,
  "cim": "Budapest, Kossuth utca 10.",
  "tipus": "Tarsashaz",
  "szobakszama": 3,
  "haviar": 180000,
  "leiras": "Felujitott lakas"
}

```

The response is a 200 OK status with a JSON body:

```

{
  "siker": true,
  "uzenet": "Ingatlan sikeresen létrehozva!"
}

```

1.Részlet az ingatlanletrehozaz.js fájlból.

Ingatlan módosítás:

Az ingatlan módosítása funkció elkészítése során lehetővé tettem a korábban rögzített ingatlanadatok szerkesztését. A fejlesztés során az API képes az adott azonosítóhoz tartozó rekord adatainak frissítésére az adatbázisban. A módosítható adatok közé tartozik többek között a cím, a típus, a szobák száma, a havi díj és a leírás. A végpont sikeres módosítás esetén visszaigazolást küld a rendszer felé. A működés helyességét Postman segítségével több tesztet végrehajtásával ellenőriztem.

```

const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

  const {
    cim,
    tipus,
    szobakszama,
    haviar,
    leiras
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `UPDATE ingatlanok
      SET
        cim = ?,
        tipus = ?,
        szobakszama = ?,
        haviar = ?,
        leiras = ?
      WHERE id = ?`,
      [
        cim,
        tipus,
        szobakszama,
        haviar,
        leiras,
        req.params.id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Ingatlan modositva!"
    });

  } catch (err) {

    res.status(500).json(err);
  }
}

```

```

    } finally {

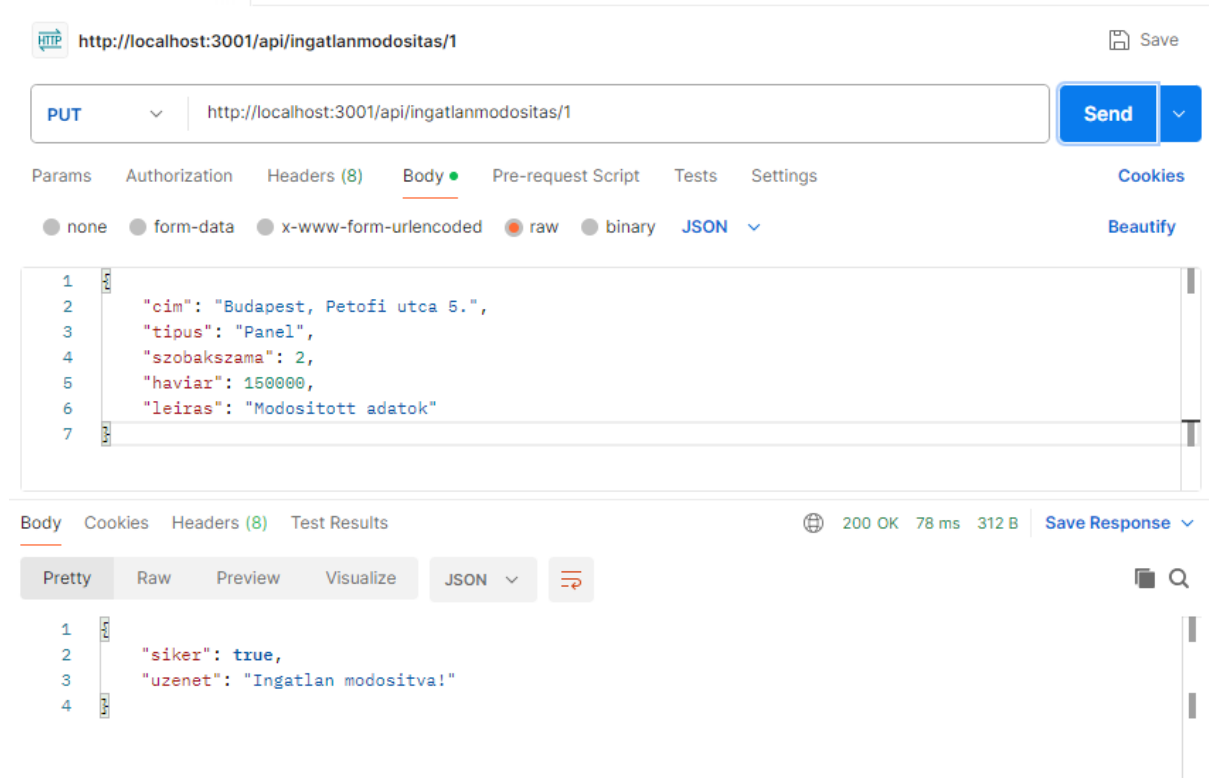
        if(conn) conn.release();

    }

});

module.exports = router;

```



2.Részlet az ingatlanmodositas.js fájlból.

Ingatlan törlés:

Az ingatlan törlése funkció fejlesztése során elkészítettem az adott ingatlan rekord eltávolítására szolgáló API végpontot. A megoldás az ingatlan azonosítója alapján törli a kiválasztott rekordot az adatbázisból. A fejlesztés során figyelmet fordítottam az adatbázis-kapcsolat megfelelő kezelésére és a hibák kezelésére is. A végpont sikeres törlés esetén tájékoztató üzenetet küld vissza a felhasználó számára. A funkció működését Postman segítségével teszteltem és sikeresen ellenőriztem.

```

const express = require("express");
const router = express.Router();

const pool = require("../db");

router.delete("/:id", async (req, res) => {

    let conn;

```

```

try {

    conn = await pool.getConnection();

    await conn.query(
        "DELETE FROM ingatlanok WHERE id = ?",
        [req.params.id]
    );

    res.json({
        siker: true,
        uzenet: "Ingatlan torolve!"
    });

} catch (err) {

    res.status(500).json(err);

} finally {

    if(conn) conn.release();

}

});

module.exports = router;

```

HTTP <http://localhost:3001/api/inglantorles/1> Save

DELETE <http://localhost:3001/api/inglantorles/1> Send

Params Authorization Headers (6) **Body** Pre-request Script Tests Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary

This request does not have a body

Body Cookies Headers (8) Test Results 200 OK 8 ms 310 B Save Response

Pretty Raw Preview Visualize JSON ⌵ ⌵

```

1  {
2    "siker": true,
3    "uzenet": "Ingatlan torolve!"
4  }

```

Ingatlan listázás:

Az ingatlanok listázása funkció fejlesztése során elkészítettem az összes rögzített ingatlan lekérdezésére szolgáló API végpontot. A végpont az adatbázisból lekéri az ingatlanok adatait, majd JSON formátumban továbbítja azokat a kliens számára. A fejlesztés célja az volt, hogy a frontend egyszerűen megjeleníthesse a rendszerben tárolt ingatlanokat. A lekérdezés során biztosítottam az adatbázis-kapcsolat megfelelő kezelését és lezárását. Az elkészült funkció működését Postman alkalmazás segítségével sikeresen teszteltem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    const adatok = await conn.query(
      "SELECT * FROM ingatlanok"
    );

    res.json(adatok);

  } catch (err) {

    res.status(500).json(err);

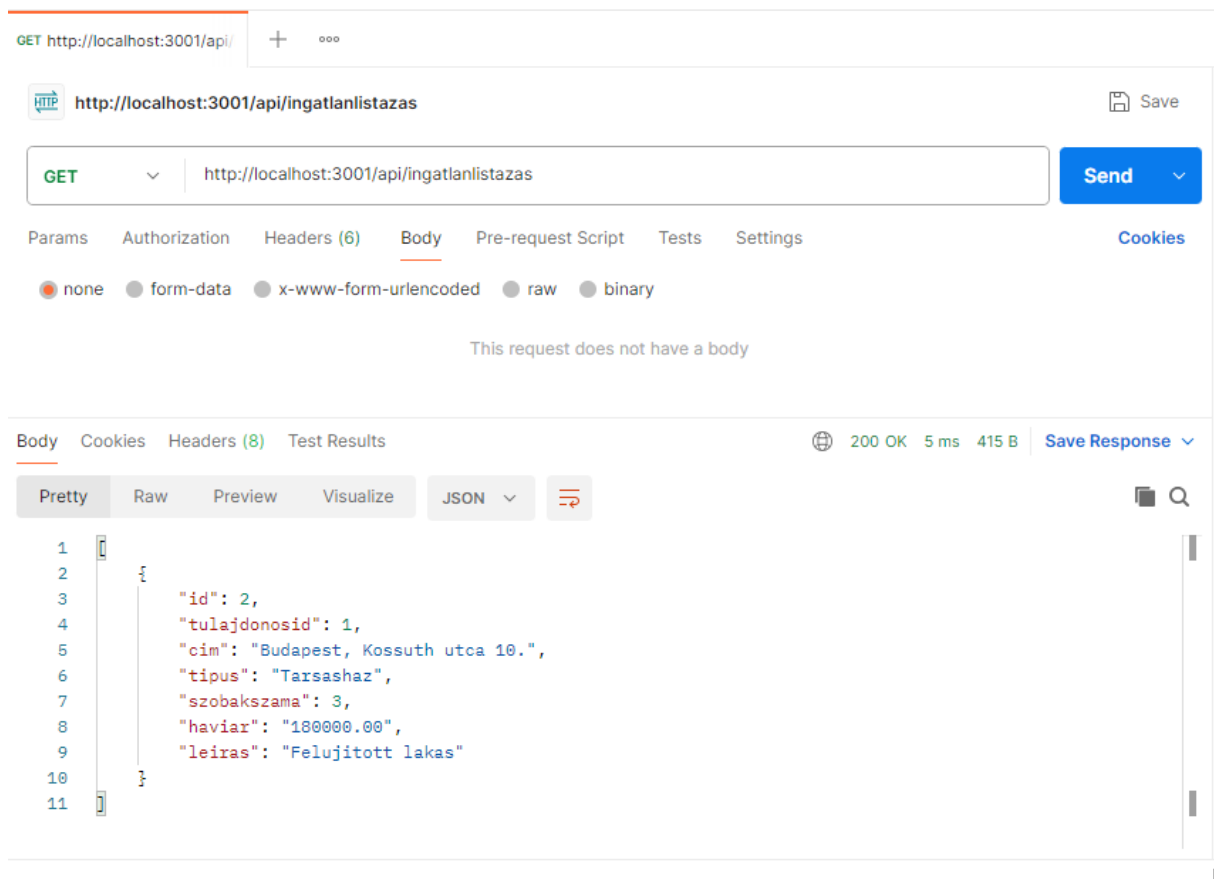
  } finally {

    if(conn) conn.release();

  }

});

module.exports = router;
```



4.Részlet az ingatlanlistazas.js fájlból.

1.7. Bérlők CRUD műveletei

A munkafolyamat során elkészítettem a bérlők kezeléséhez kapcsolódó backend funkciókat az Ingatlankezelő rendszerben. Fejlesztettem a bérlők létrehozására, módosítására, törlésére és listázására szolgáló API végpontokat, amelyek lehetővé teszik a bérleti adatok kezelését az adatbázisban. Megvalósítottam továbbá a bérlők és ingatlanok összekapcsolását biztosító funkciót, amely új bérleti szerződés létrehozásával kezeli a hozzárendelést. A fejlesztés során SQL lekérdezéseket alkalmaztam az adatok rögzítésére, módosítására, törlésére és lekérdezésére. Külön figyelmet fordítottam az adatbázis-kapcsolatok kezelésére és a hibakezelés megfelelő megvalósítására. Az elkészült végpontokat Postman alkalmazás segítségével teszteltem, ahol ellenőriztem az adatok helyes mentését, módosítását, törlését és megjelenítését az adatbázisban.

Bérlők létrehozás:

A bérliő létrehozása funkció fejlesztése során elkészítettem az új bérliők adatainak rögzítésére szolgáló API végpontot. A rendszer képes eltárolni a bérliő személyes adatait, például nevét, email címét, telefonszámát és lakcímét. A fejlesztés során kialakítottam az adatbázisba történő adatmentés folyamatát és a szükséges SQL utasításokat. A végpont sikeres mentés esetén visszajelzést küld a kliens oldal számára. A működést Postman segítségével ellenőriztem és teszteltem.

```

const express = require("express");
const router = express.Router();

```

```

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    userid,
    nev,
    email,
    telefonszam,
    személyiszam,
    lakcim
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `INSERT INTO berlok
      (userid, nev, email, telefonszam, személyiszam, lakcim)
      VALUES (?, ?, ?, ?, ?, ?)` ,
      [
        userid,
        nev,
        email,
        telefonszam,
        személyiszam,
        lakcim
      ]
    );

    res.json({
      siker: true,
      uzenet: "Berlo sikeresen letrehozva!"
    });

  } catch (err) {

    res.status(500).json(err);

  } finally {

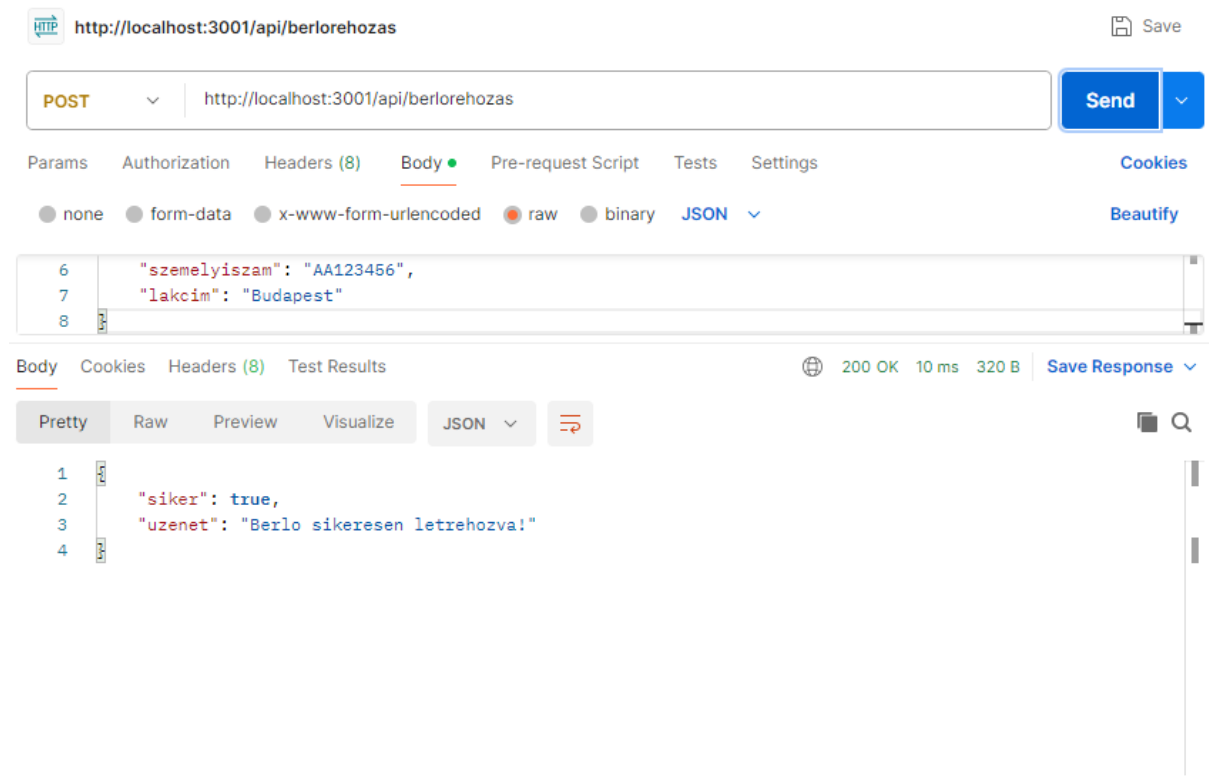
    if(conn) conn.release();

  }
}

```

```
});
```

```
module.exports = router;
```



1. Részlet az *berloletrehoz.js* fájlból.

Bérlők módosítása:

A bérlő módosítása funkció elkészítése során lehetővé tettem a bérlő adatainak frissítését az adatbázisban. A rendszer képes a kiválasztott bérlő személyes adatainak módosítására és mentésére. A fejlesztés során létrehoztam az adatbázis frissítéséhez szükséges SQL lekérdezéseket. A módosítás sikeres végrehajtása esetén a rendszer visszajelzést küld a felhasználó számára. A funkció helyes működését Postman tesztekkel ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

  const {
    nev,
    email,
    telefonszam,
    személyiszam,
    lakcim
```

```

    } = req.body;

    let conn;

    try {

        conn = await pool.getConnection();

        await conn.query(
            `UPDATE berlok
            SET
            nev = ?,
            email = ?,
            telefonszam = ?,
            személyiszam = ?,
            lakcim = ?
            WHERE id = ?`,
            [
                nev,
                email,
                telefonszam,
                személyiszam,
                lakcim,
                req.params.id
            ]
        );

        res.json({
            siker: true,
            uzenet: "Berlo modositva!"
        });

    } catch (err) {

        res.status(500).json(err);

    } finally {

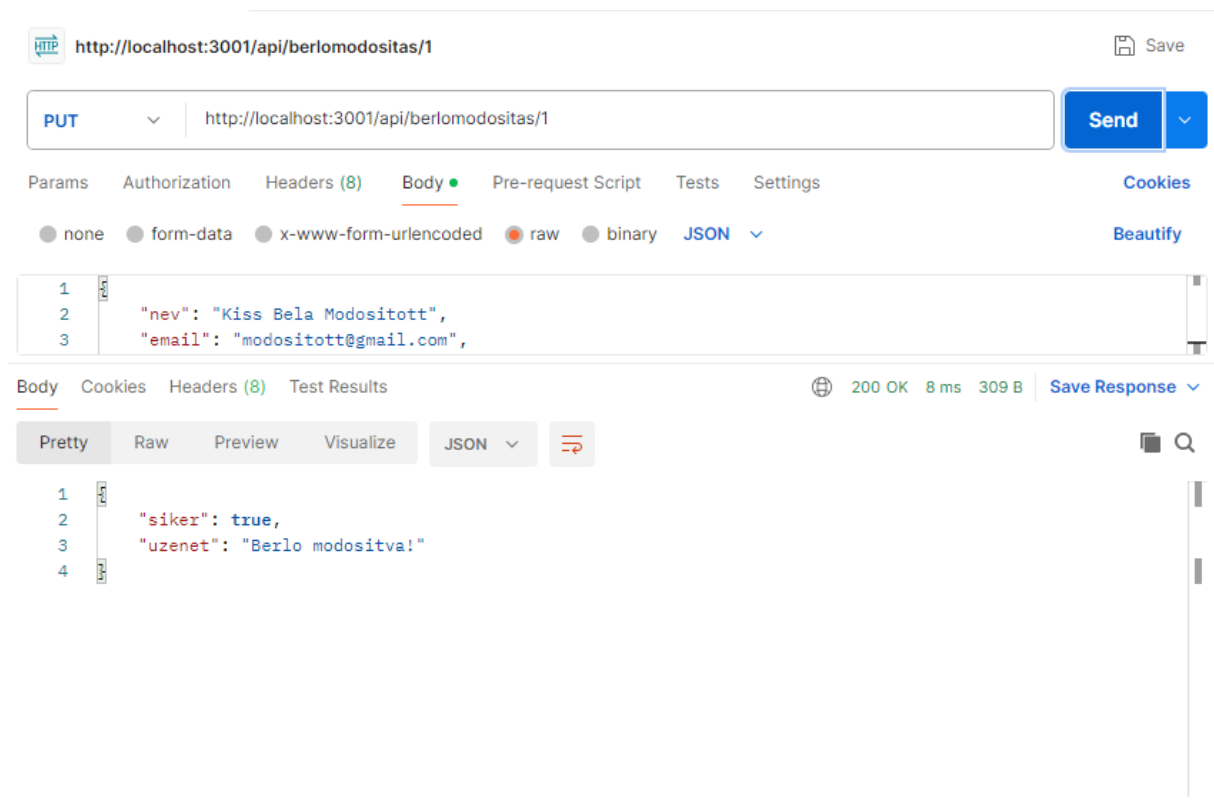
        if(conn) conn.release();

    }

});

module.exports = router;

```



1. Részlet az berlomodositas.js fájlból.

Bérlők törlése:

A bérlő törlése funkció fejlesztése során elkészítettem a kiválasztott bérlő rekordjának eltávolítására szolgáló API végpontot. A rendszer az azonosító alapján megkeresi és törli a megfelelő adatot az adatbázisból. A fejlesztés során figyelmet fordítottam a biztonságos adatkezelésre és a hibakezelésre is. Sikeres törlés esetén a végpont tájékoztató üzenetet küld vissza a kliensnek. A funkció működését Postman alkalmazás segítségével teszteltem és ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.delete("/:id", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      "DELETE FROM berlok WHERE id = ?",
      [req.params.id]
    );

  };
```

```

        res.json({
            siker: true,
            uzenet: "Berlo torolve!"
        });

    } catch (err) {

        res.status(500).json(err);

    } finally {

        if(conn) conn.release();

    }

});

module.exports = router;

```

The screenshot shows a web browser's developer tools interface. At the top, there's a search bar with the URL `http://localhost:3001/api/berlotorles/1` and a `DELETE` method selected. Below this, there are tabs for `Params`, `Authorization`, `Headers (6)`, `Body`, `Pre-request Script`, `Tests`, and `Settings`. The `Body` tab is active, showing a message: "This request does not have a body". Below the request section, there's a section for the response. It shows the `Body` tab selected, with a `JSON` view of the response: `{siker: true, uzenet: 'Berlo torolve!'}`. The status bar at the bottom indicates a `200 OK` status, a response time of `10 ms`, and a response size of `307 B`.

3. Részlet az `berlotorles.js` fájlból.

Bérlők listázás:

A bérlők listázása funkció fejlesztése során elkészítettem az összes bérlő adatainak lekérdezésére szolgáló API végpontot. A rendszer az adatbázisból lekéri a bérlők adatait és JSON formátumban továbbítja azokat a frontend számára. A fejlesztés célja az volt, hogy a felhasználók egyszerűen áttekinthető listában jeleníthessék meg a bérlőket. A lekérdezés során

biztosítottam az adatbázis-kapcsolat megfelelő kezelését. Az elkészült végpont működését Postman segítségével sikeresen teszteltem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    const adatok = await conn.query(
      "SELECT * FROM berlok"
    );

    res.json(adatok);

  } catch (err) {

    res.status(500).json(err);

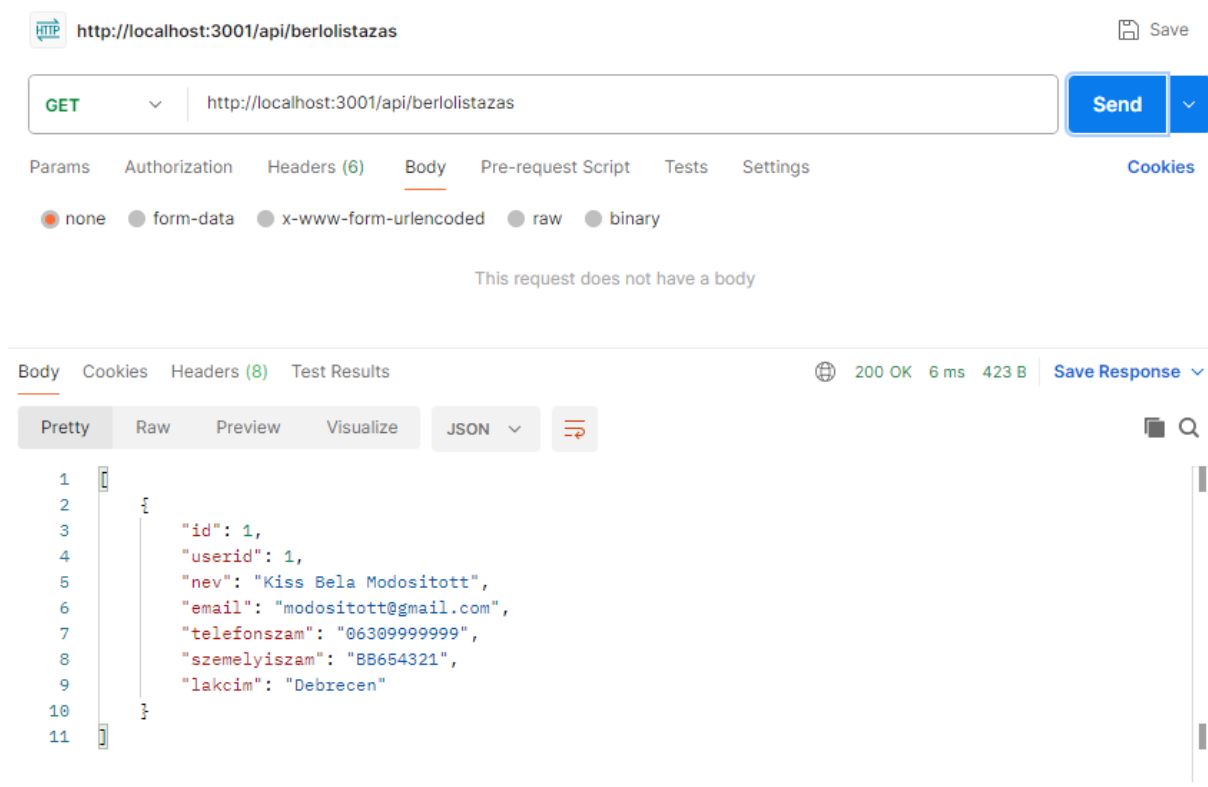
  } finally {

    if(conn) conn.release();

  }

});

module.exports = router;
```



4. Részlet az `berlolistazas.js` fájlból.

Bérlő hozzárendelése ingatlanhoz:

A bérlő hozzárendelése funkció fejlesztése során elkészítettem azt az API végpontot, amely lehetővé teszi egy bérlő és egy ingatlan összekapcsolását a rendszerben. A megvalósítás során a hozzárendelés egy új bérleti szerződés létrehozásával történt, amely tartalmazza az ingatlan és a bérlő azonosítóját, valamint a szerződéshez kapcsolódó adatokat. A rendszer eltárolja a szerződés kezdeti és végdátumát, a havi bérleti díjat és a szerződés aktuális állapotát is. A fejlesztés során biztosítottam, hogy a hozzárendelés csak létező bérlő és ingatlan rekordok között történhessen meg az adatbázis idegen kulcs kapcsolatain keresztül. Az elkészült funkció segítségével nyilvántarthatóvá vált, hogy melyik bérlő melyik ingatlanhoz tartozik. A végpont működését Postman alkalmazás segítségével teszteltem, ahol ellenőriztem az adatok sikeres rögzítését és az adatbázisban történő megjelenését.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    ingatlanid,
    berloid,
    kezdetdatum,
    vegdatum,
    havidij,
```

```

        statusz
    } = req.body;

    let conn;

    try {

        conn = await pool.getConnection();

        await conn.query(
            `INSERT INTO szerzodesek
            (ingatlanid, berloid, kezdetdatum, vegdatum, havidij, statusz)
            VALUES (?, ?, ?, ?, ?, ?)` ,
            [
                ingatlanid,
                berloid,
                kezdetdatum,
                vegdatum,
                havidij,
                statusz
            ]
        );

        res.json({
            siker: true,
            uzenet: "Bérlő sikeresen hozzárendelve az ingatlanhoz!"
        });

    } catch (err) {

        res.status(500).json(err);

    } finally {

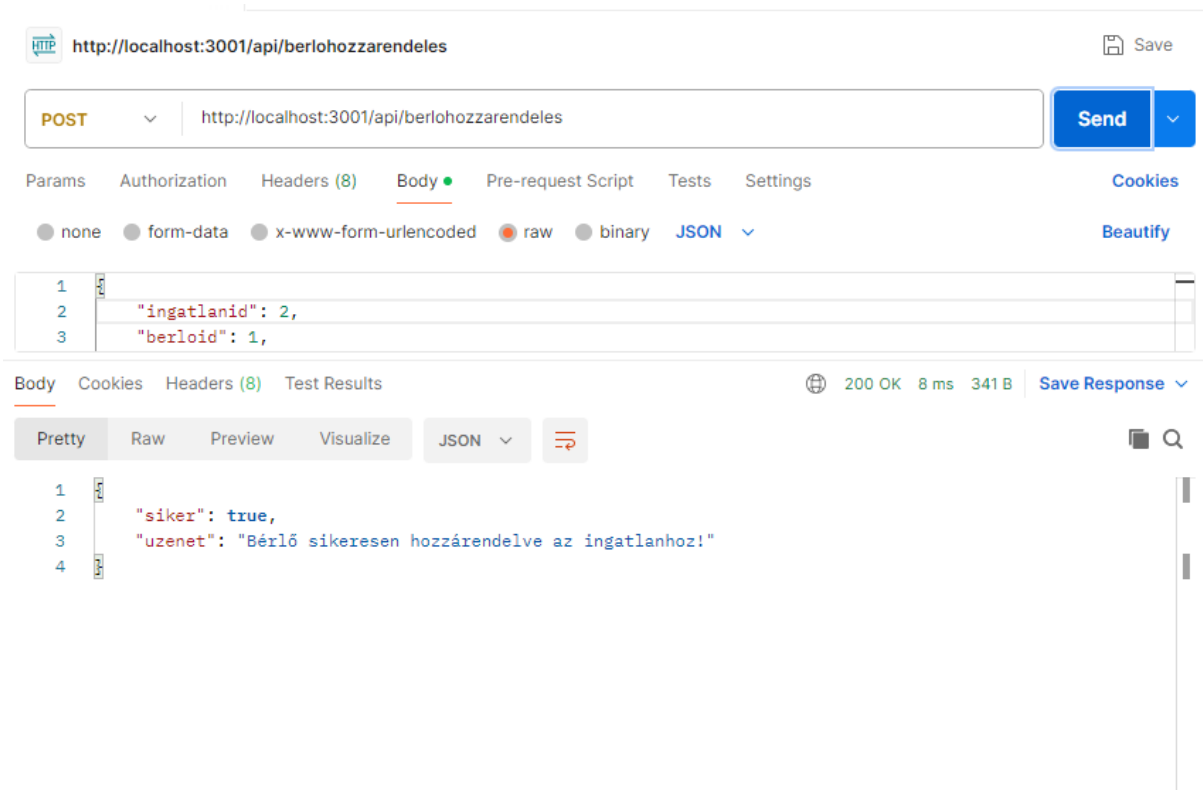
        if(conn) conn.release();

    }

});

module.exports = router;

```



5. Részlet az berlohozzarendeles.js fájlból.

1.8 Fizetések CRUD műveletei

A munkafolyamat során elkészítettem a fizetések kezeléséhez kapcsolódó backend funkciókat az Ingatlankezelő rendszerben. Fejlesztettem a fizetések létrehozására, módosítására, törlésére és listázására szolgáló API végpontokat, amelyek lehetővé teszik a befizetések adatainak kezelését az adatbázisban. A fejlesztés során kialakítottam az adatok rögzítéséhez, módosításához és lekérdezéséhez szükséges SQL utasításokat, valamint biztosítottam az adatbázis-kapcsolatok megfelelő kezelését. A rendszer képes eltárolni a befizetés összegét, dátumát, státuszát, fizetési módját és a hozzá tartozó megjegyzéseket is. Külön figyelmet fordítottam a hibakezelésre és a sikeres műveletek visszajelzésére a kliens oldal számára. Az elkészült végpontokat Postman alkalmazás segítségével teszteltem, ahol ellenőriztem a fizetések létrehozását, módosítását, törlését és listázását, valamint az adatok helyes megjelenését az adatbázisban.

Fizetés létrehozása:

A fizetés létrehozása funkció fejlesztése során elkészítettem az új befizetések rögzítésére szolgáló API végpontot. A rendszer eltárolja a szerződés azonosítóját, a befizetett összeget, a fizetés dátumát, státuszát és fizetési módját. A fejlesztés során kialakítottam az adatok adatbázisba történő mentésének folyamatát. A végpont sikeres adatmentés esetén visszajelzést küld a kliens számára. A működés helyességét Postman alkalmazással ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {
```

```

const {
  szerzodesid,
  osszeg,
  fizetesdatum,
  statusz,
  fizetesmod,
  megjegyzes
} = req.body;

let conn;

try {

  conn = await pool.getConnection();

  await conn.query(
    `INSERT INTO fizetesek
    (szerzodesid, osszeg, fizetesdatum, statusz, fizetesmod, megjegyzes)
    VALUES (?, ?, ?, ?, ?, ?)`,
    [
      szerzodesid,
      osszeg,
      fizetesdatum,
      statusz,
      fizetesmod,
      megjegyzes
    ]
  );

  res.json({
    siker: true,
    uzenet: "Fizetes sikeresen rogzitve!"
  });

} catch (err) {

  res.status(500).json(err);

} finally {

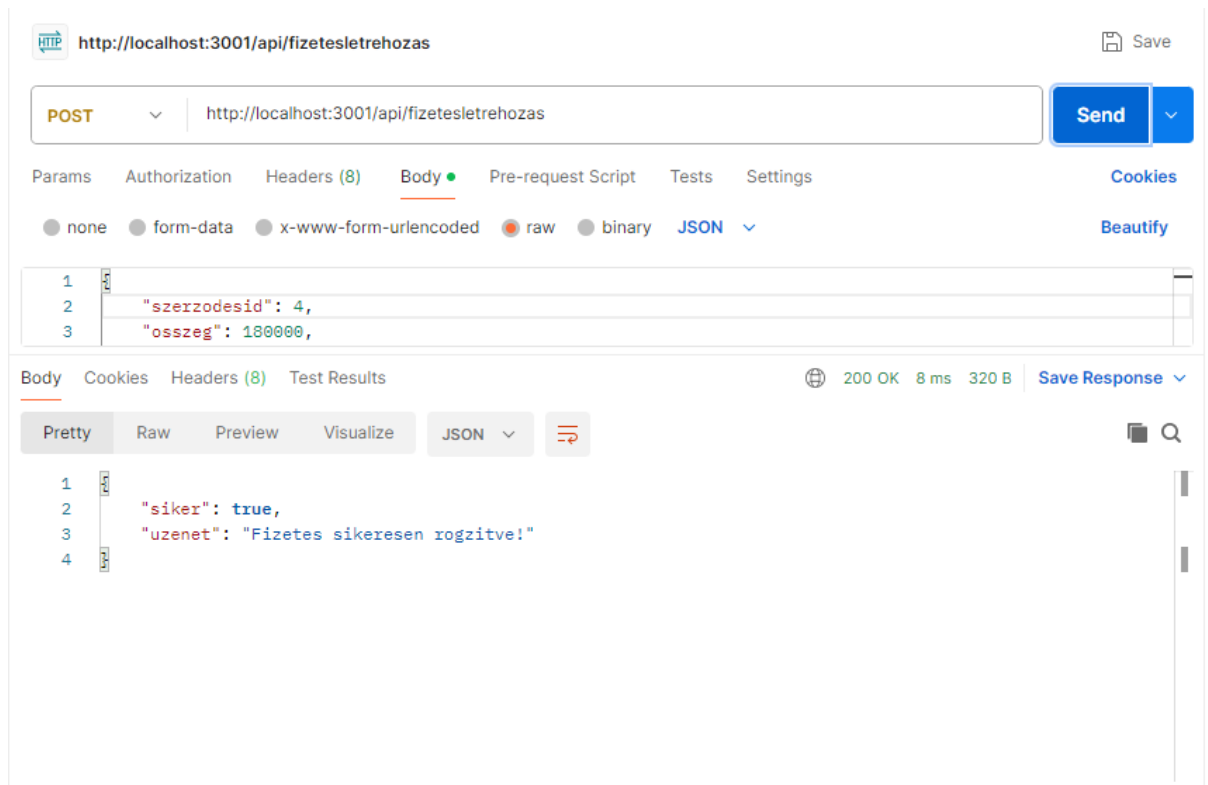
  if(conn) conn.release();

}

});

module.exports = router;

```



1. Részlet az `fizetesletrehozaz.js` fájlból.

Fizetés módosítása:

A fizetés módosítása funkció elkészítése során lehetővé tettem a már rögzített fizetési adatok szerkesztését. A rendszer képes módosítani a befizetett összeget, a fizetés dátumát, státuszát, fizetési módját és a megjegyzéseket. A fejlesztés során elkészítettem a szükséges SQL frissítő lekérdezéseket és backend logikát. A sikeres módosítás után a rendszer visszajelzést küld a felhasználó számára. A működés helyességét Postman segítségével ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

  const {
    osszeg,
    fizetesdatum,
    statusz,
    fizetesmod,
    megjegyzes
  } = req.body;

  let conn;

  try {
```

```

    conn = await pool.getConnection();

    await conn.query(
      `UPDATE fizeteselek
      SET
        osszeg = ?,
        fizetesdatum = ?,
        statusz = ?,
        fizetesmod = ?,
        megjegyzes = ?
      WHERE id = ?`,
      [
        osszeg,
        fizetesdatum,
        statusz,
        fizetesmod,
        megjegyzes,
        req.params.id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Fizetes modositva!"
    });

  } catch (err) {

    res.status(500).json(err);

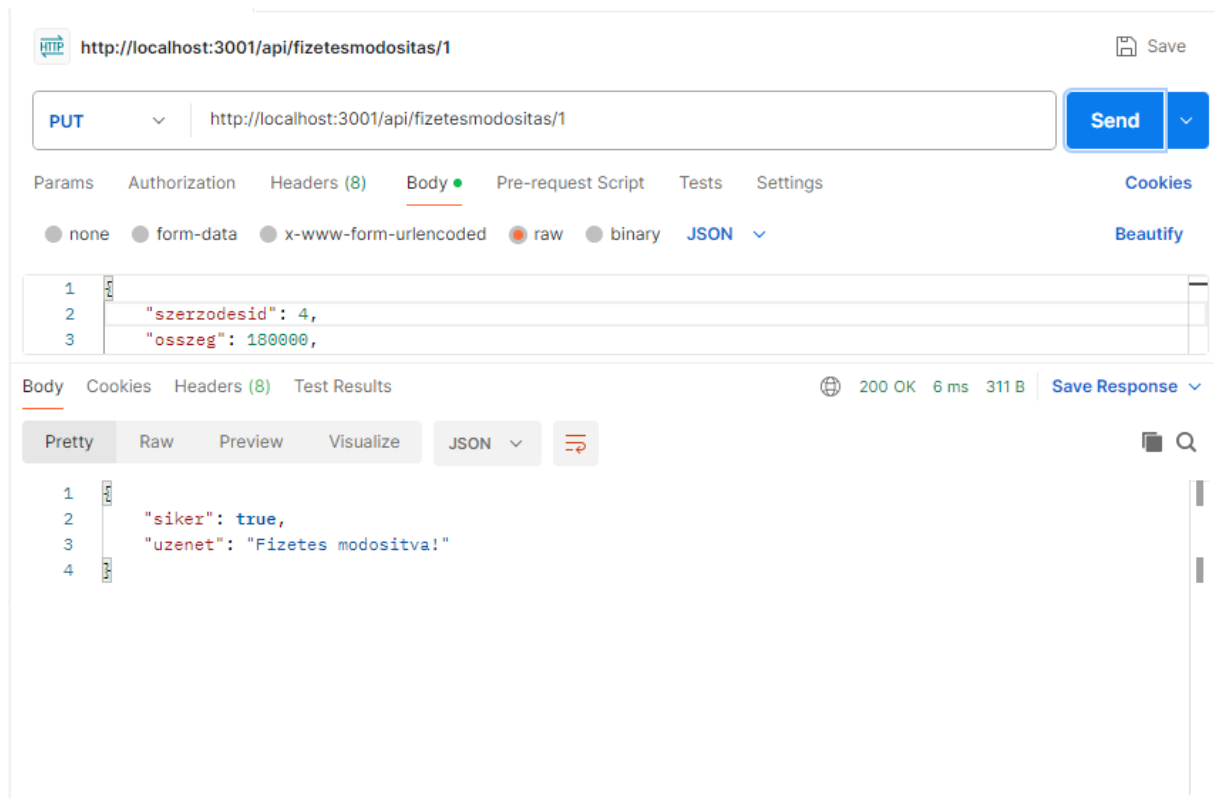
  } finally {

    if(conn) conn.release();

  }
});

module.exports = router;

```



2. Részlet az `fizetesmodositas.js` fájlból.

Fizetés törlése:

A fizetés törlése funkció fejlesztése során elkészítettem a fizetési rekordok eltávolítására szolgáló API végpontot. A rendszer a megadott azonosító alapján törli a kiválasztott befizetést az adatbázisból. A fejlesztés során biztosítottam a hibák megfelelő kezelését és az adatbázis-kapcsolat szabályos lezárását. A sikeres törlést követően a rendszer tájékoztató üzenetet küld vissza a kliens oldal számára. Az elkészült funkció működését Postman alkalmazás segítségével teszteltem és ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.delete("/:id", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      "DELETE FROM fizetesek WHERE id = ?",
      [req.params.id]
    );

  }
```

```

        res.json({
            siker: true,
            uzenet: "Fizetes torolve!"
        });

    } catch (err) {

        res.status(500).json(err);

    } finally {

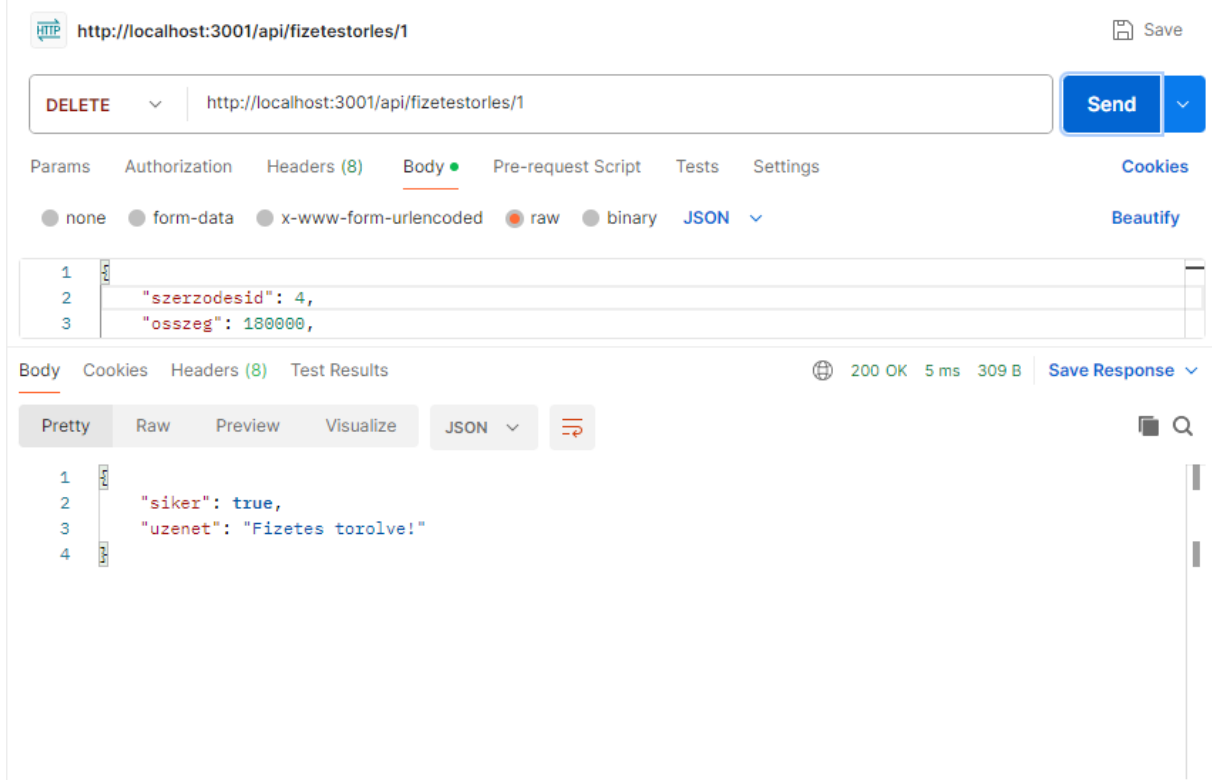
        if(conn) conn.release();

    }

});

module.exports = router;

```



3.Részlet az `fizetestorles.js` fájlból.

Fizetés listázása:

A fizetések listázása funkció fejlesztése során elkészítettem az összes befizetés lekérdezésére szolgáló API végpontot. A rendszer az adatbázisból lekéri a fizetésekhez tartozó adatokat, majd JSON formátumban visszaküldi azokat. A fejlesztés célja az volt, hogy a felhasználók egyszerűen áttekinthető formában láthassák a befizetések adatait. A lekérdezések során biztosítottam az adatbázis-kapcsolat megfelelő kezelését. Az elkészült funkciót Postman segítségével teszteltem.

```
const express = require("express");
```

```
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    const adatok = await conn.query(
      "SELECT * FROM fizetesesek"
    );

    res.json(adatok);

  } catch (err) {

    res.status(500).json(err);

  } finally {

    if(conn) conn.release();

  }

});

module.exports = router;
```

1.9 Szerződések CRUD műveletei

A munkafolyamat során elkészítettem a szerződések kezeléséhez kapcsolódó backend funkciókat az Ingatlankezelő rendszerben. Fejlesztettem a szerződések létrehozására, módosítására, törlésére és listázására szolgáló API végpontokat, amelyek lehetővé teszik a bérleti szerződések teljes körű kezelését. A rendszer képes kezelni az ingatlanok és bérlők összekapcsolását, valamint a szerződésekhez tartozó kezdő- és végdátumokat, havi díjakat és státuszokat. A fejlesztés során SQL INSERT, UPDATE, DELETE és SELECT lekérdezéseket alkalmaztam az adatok kezelésére. Külön figyelmet fordítottam az adatbázis-kapcsolatok megfelelő kezelésére, a hibakezelésre és a sikeres műveletek visszajelzésére. Az elkészült végpontokat Postman alkalmazás segítségével teszteltem, ahol ellenőriztem a szerződések létrehozását, módosítását, törlését és listázását, valamint az adatok helyes tárolását az adatbázisban.

Szerződés létrehozása:

A szerződés létrehozása funkció fejlesztése során elkészítettem azt az API végpontot, amely új bérleti szerződések rögzítését teszi lehetővé az adatbázisban. A végpont fogadja az ingatlan és a bérelő azonosítóját, valamint a szerződéshez kapcsolódó adatokat, például a kezdő- és végdátumot, a havi díjat és a státuszt. A fejlesztés során SQL INSERT utasítással valósítottam meg az adatok eltárolását. A sikeres művelet után a rendszer visszajelzést küld a felhasználó számára. Az elkészült funkció működését Postman alkalmazással teszteltem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    ingatlanid,
    berloid,
    kezdetdatum,
    vegdatum,
    havidij,
    statusz
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `INSERT INTO szerzodesek
      (ingatlanid, berloid, kezdetdatum, vegdatum, havidij, statusz)
      VALUES (?, ?, ?, ?, ?, ?)` ,
      [
        ingatlanid,
        berloid,
        kezdetdatum,
        vegdatum,
        havidij,
        statusz
      ]
    );

    res.json({
      siker: true,
      uzenet: "Szerzodes sikeresen letrehozva!"
    });
  }
```

```

    } catch (err) {

        res.status(500).json(err);

    } finally {

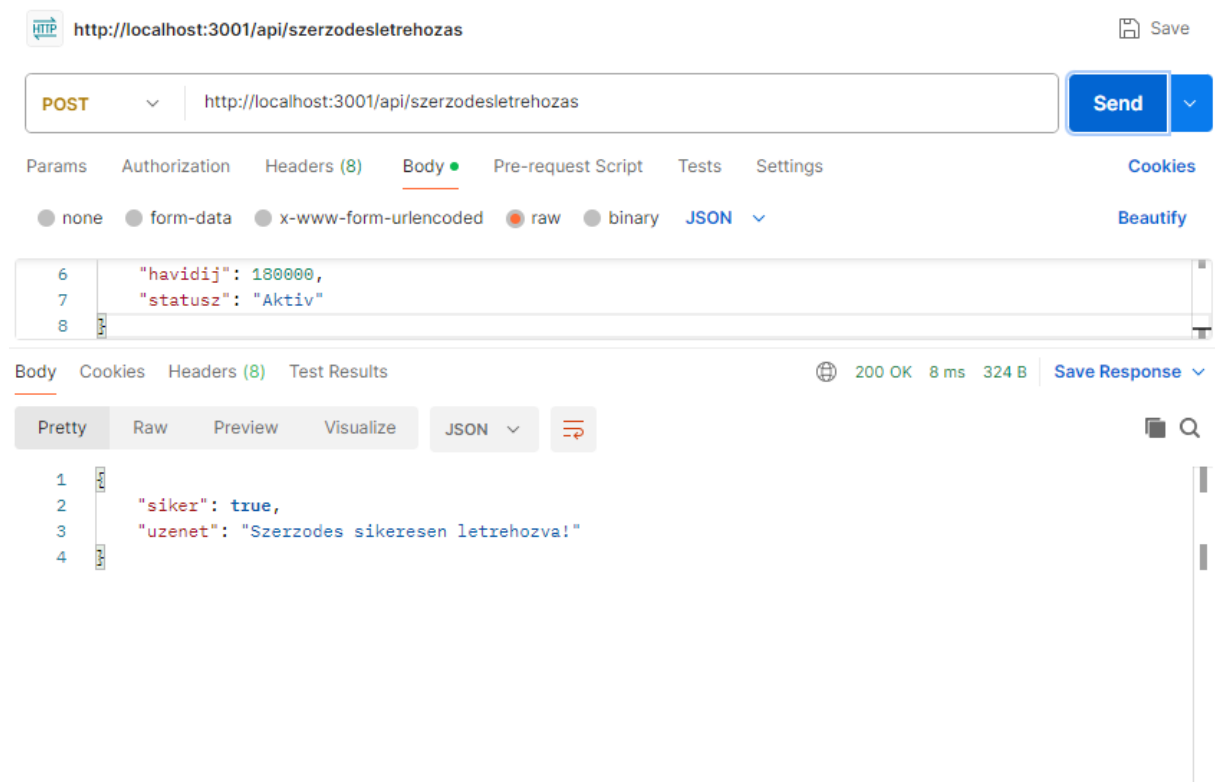
        if(conn) conn.release();

    }

});

module.exports = router;

```



1.Részlet a szerzodesletrehozaz.js fájlból.

Szerződés módosítása:

A szerződés módosítása funkció elkészítése során lehetővé tettem a meglévő szerződések adatainak frissítését. A rendszer képes módosítani a szerződés kezdő- és végdátumát, havi díját és státuszát. A fejlesztés során SQL UPDATE lekérdezést alkalmaztam az adatok frissítésére. A sikeres módosítás után a rendszer visszaigazoló üzenetet küld. A működés helyességét Postman alkalmazással teszteltem.

```

const express = require("express");
const router = express.Router();

const pool = require("../db");

```

```

router.put("/:id", async (req, res) => {

  const {
    kezdetdatum,
    vegdatum,
    havidij,
    statusz
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `UPDATE szerzodesek
      SET
        kezdetdatum = ?,
        vegdatum = ?,
        havidij = ?,
        statusz = ?
      WHERE id = ?`,
      [
        kezdetdatum,
        vegdatum,
        havidij,
        statusz,
        req.params.id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Szerzodes modositva!"
    });

  } catch (err) {

    res.status(500).json(err);

  } finally {

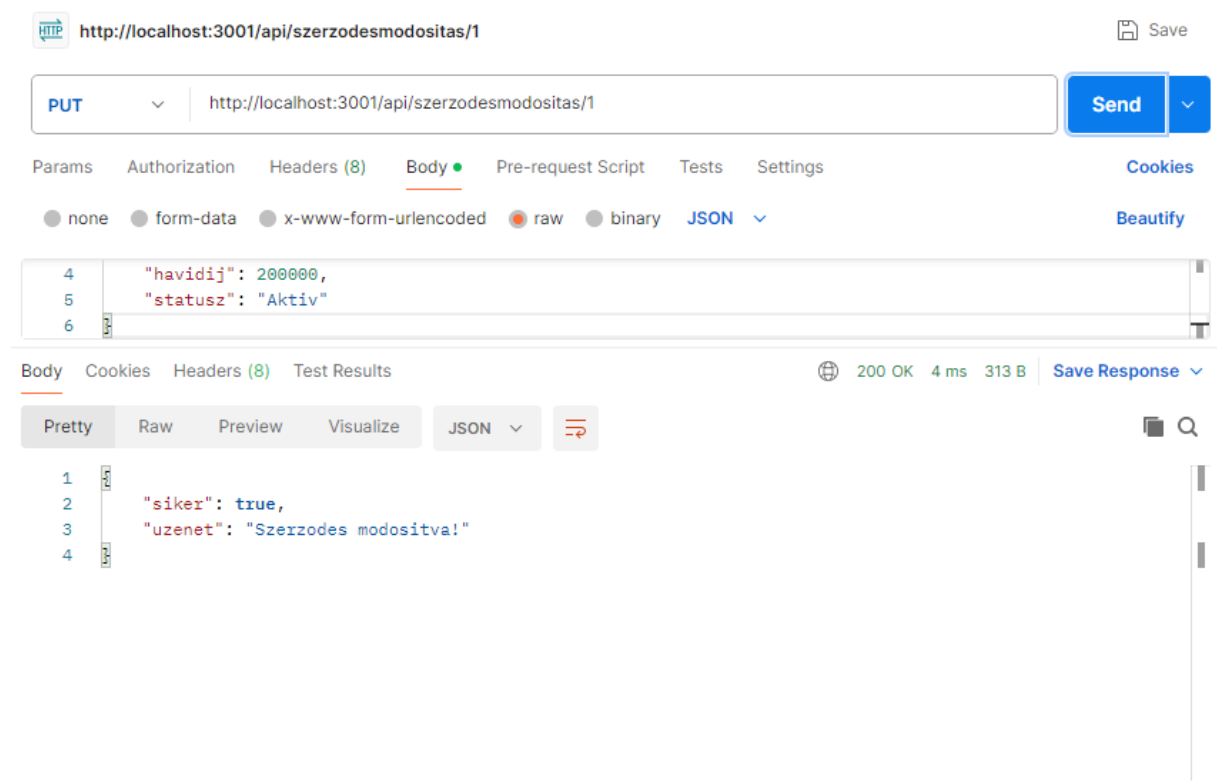
    if(conn) conn.release();

  }

});

```

```
module.exports = router;
```



2.Részlet a szerzodesmodositas.js fájlból.

Szerződés törlése:

A szerződés törlése funkció fejlesztése során elkészíttem a szerződések eltávolítására szolgáló API végpontot. A rendszer a megadott azonosító alapján törli a kiválasztott szerződést az adatbázisból. A fejlesztés során figyelmet fordítottam a hibakezelésre és az adatbázis-kapcsolat megfelelő lezárására. A sikeres törlés után a rendszer visszajelzést küld a felhasználónak. Az elkészült funkció működését Postman segítségével ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.delete("/:id", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      "DELETE FROM szerzodesek WHERE id = ?",
      [req.params.id]
    );

  };
```

```

        res.json({
            siker: true,
            uzenet: "Szerződés törölve!"
        });

    } catch (err) {

        res.status(500).json(err);

    } finally {

        if(conn) conn.release();

    }

});

module.exports = router;

```

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3001/api/szerzodestorles/1`
- Method:** DELETE
- Body:**

```

4  "havidij": 200000,
5  "statusz": "Aktiv"
6

```
- Response:**

```

1  {
2    "siker": true,
3    "uzenet": "Szerződés törölve!"
4  }

```
- Status:** 200 OK, 4 ms, 311 B

3.Részlet a szerzodestorles.js fájlból.

Szerződés listázása:

A szerződések listázása funkció fejlesztése során elkészítettem az összes rögzített szerződés lekérdezésére szolgáló API végpontot. A rendszer az adatbázisból lekéri a szerződések adatait és JSON formátumban továbbítja azokat a kliens számára. A fejlesztés célja az volt, hogy a felhasználók egyszerűen áttekinthető formában láthassák a szerződéseket. A lekérdezés során

biztosítottam az adatbázis-kapcsolat megfelelő kezelését. A funkció helyes működését Postman segítségével ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    const adatok = await conn.query(
      "SELECT * FROM szerzodesek"
    );

    res.json(adatok);

  } catch (err) {

    res.status(500).json(err);

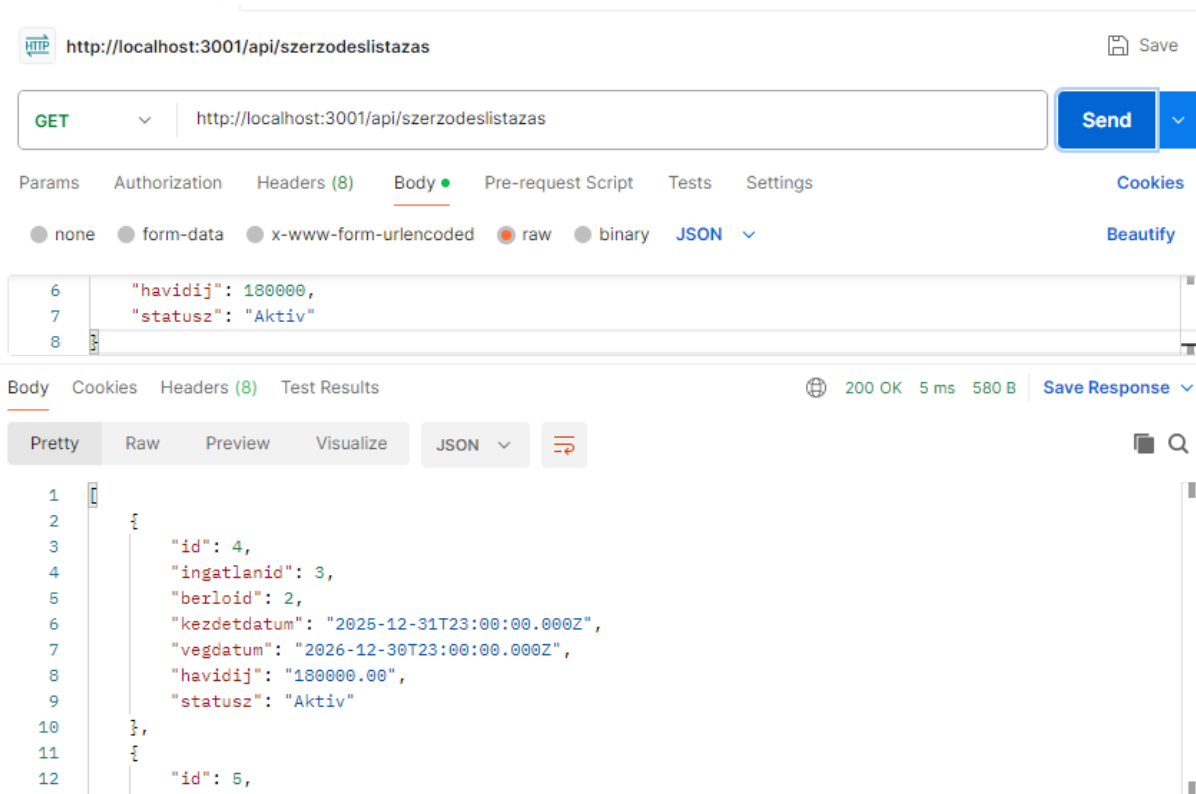
  } finally {

    if(conn) conn.release();

  }

});

module.exports = router;
```



4. Részlet a szerzodeslistazas.js fájlból.

1.10. Üzenetek CRUD műveletei

A munkafolyamat során elkészítettem az üzenetek kezeléséhez kapcsolódó backend funkciókat az Ingatlankezelő rendszerben. Fejlesztettem egy API végpontot, amely lehetővé teszi a bérlők számára üzenetek küldését az adminisztrátor részére, valamint egy másik végpontot az üzenetek lekérdezésére és megjelenítésére. Az üzenetek küldése során a rendszer eltárolja a küldő azonosítóját, az üzenet tárgyát és tartalmát az adatbázisban. Az adminisztrátor számára biztosított listázó funkció lehetővé teszi a beérkezett üzenetek áttekintését és nyomon követését. A fejlesztés során SQL INSERT és SELECT lekérdezéseket alkalmaztam, valamint gondoskodtam a hibakezelés és az adatbázis-kapcsolatok megfelelő kezeléséről. Az elkészült végpontokat Postman alkalmazás segítségével teszteltem, ahol ellenőriztem az üzenetek sikeres rögzítését és lekérdezését az adatbázisból.

Üzenetek küldése

Az üzenetküldés funkció fejlesztése során elkészítettem azt az API végpontot, amely lehetővé teszi a bérlők számára üzenetek küldését a rendszerben. A végpont fogadja a küldő azonosítóját, az üzenet tárgyát és az üzenet szövegét, majd eltárolja azokat az adatbázisban. A fejlesztés során SQL INSERT utasítást alkalmaztam az adatok rögzítésére. Sikeres üzenetküldés esetén a rendszer visszaigazoló üzenetet küld a felhasználó számára. Az elkészült funkció működését Postman alkalmazás segítségével teszteltem és ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {
```

```

const {
  kuldoid,
  targy,
  uzenet
} = req.body;

let conn;

try {

  conn = await pool.getConnection();

  await conn.query(
    `INSERT INTO uzenetek
    (kuldoid, targy, uzenet)
    VALUES (?, ?, ?)`,
    [
      kuldoid,
      targy,
      uzenet
    ]
  );

  res.json({
    siker: true,
    uzenet: "Uzenet elkuldve!"
  });

} catch (err) {

  res.status(500).json(err);

} finally {

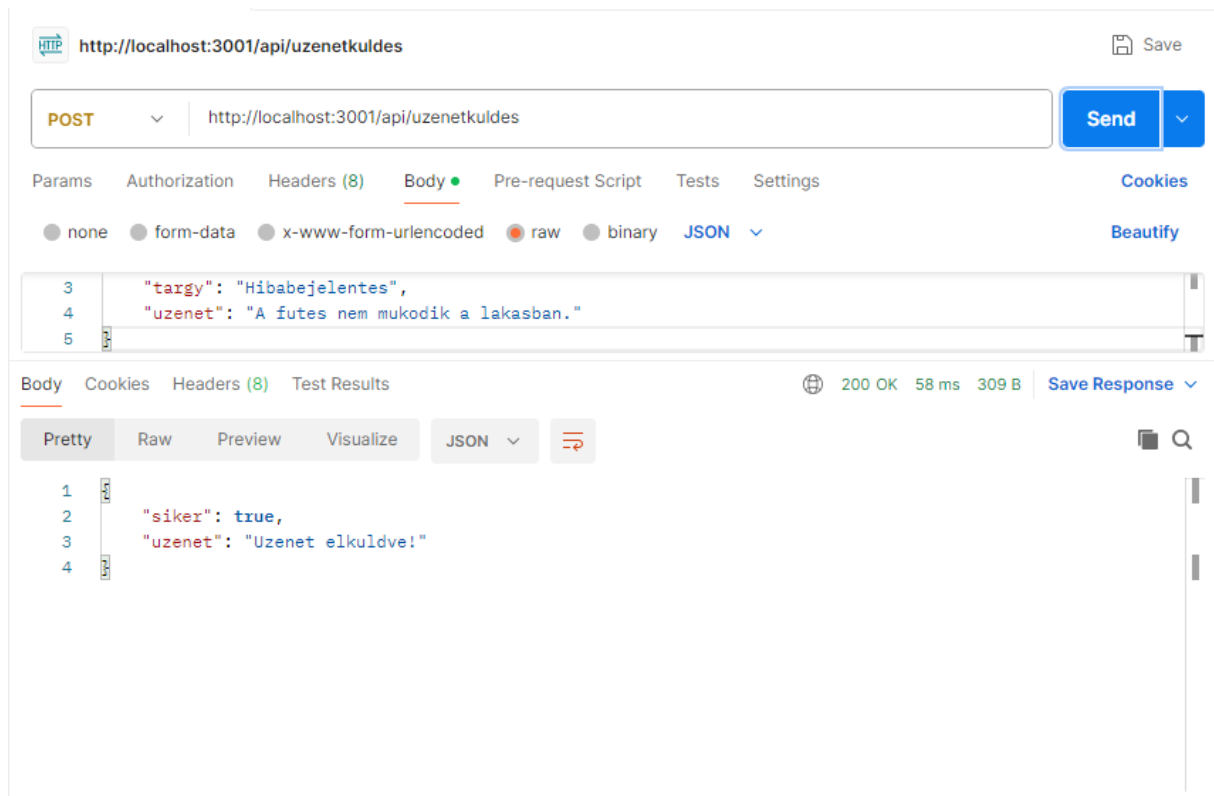
  if (conn) conn.release();

}

});

module.exports = router;

```



1. Részlet a uzenetkuldes.js fájlból.

Üzenetek listázása

Az üzenetek listázása funkció fejlesztése során elkészítettem azt az API végpontot, amely lehetővé teszi az adminisztrátor számára a rendszerben tárolt üzenetek megtekintését. A végpont lekérdezi az adatbázisból az összes rögzített üzenetet, majd JSON formátumban visszaküldi azokat a kliens számára. A fejlesztés során biztosítottam, hogy az üzenetek a küldés időpontja szerint rendezve jelenjenek meg. A funkció célja az adminisztrációs felület információellátásának támogatása és a felhasználói megkeresések nyomon követése volt. Az elkészült végpont helyes működését Postman alkalmazással teszteltem és sikeresen ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    const adatok = await conn.query(
      "SELECT * FROM uzenetek ORDER BY kuldesdatum DESC"
    );

  }
```

```

        res.json(adatok);

    } catch (err) {

        res.status(500).json(err);

    } finally {

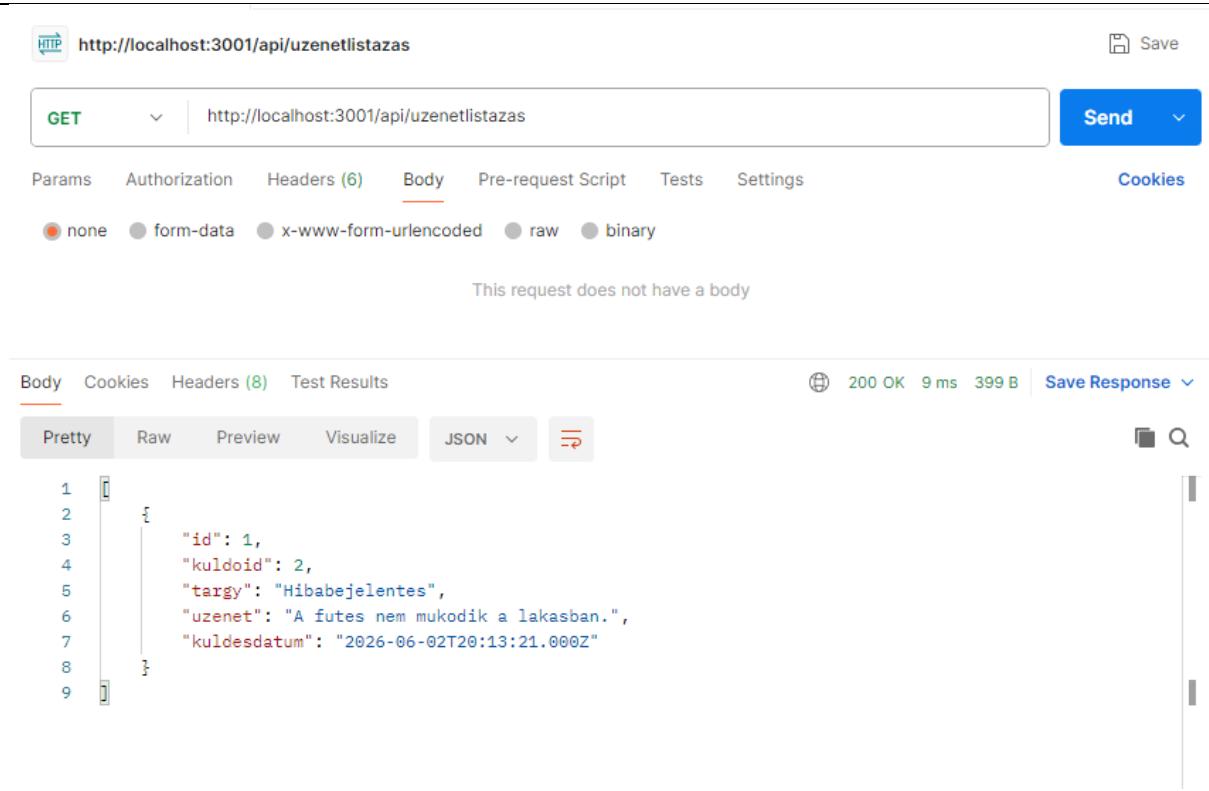
        if (conn) conn.release();

    }

});

module.exports = router;

```



The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3001/api/uzenetlistazas`
- Method:** `GET`
- Body:** This request does not have a body.
- Response:**
 - Status:** 200 OK
 - Time:** 9 ms
 - Size:** 399 B
 - Save Response:** (button)
- Response Body (JSON):**

```

{
  "id": 1,
  "kuldoid": 2,
  "targy": "Hibabejelentés",
  "uzenet": "A fűtés nem működik a lakásban.",
  "kuldesdatum": "2026-06-02T20:13:21.000Z"
}

```

2. Részlet a `uzenetlistazas.js` fájlból.

1.11 Jogosultságkezelés és admin funkciók

A jogosultságkezelés kialakítása során adminisztrátor és felhasználó szerepköröket alkalmaztam. A felhasználó szerepköre a bejelentkezés során lekérdezésre kerül az adatbázisból, majd a frontend alkalmazás ennek alapján jeleníti meg a megfelelő felhasználói felületet. Az adminisztrátorok számára elérhetővé válnak az adatkezelési funkciók, míg a felhasználók kizárólag a számukra szükséges műveleteket érhetik el. A megoldás biztosítja, hogy a különböző szerepkörök eltérő funkcionálisokhoz férjenek hozzá a rendszerben.

Regisztráció API elkészítése

A regisztrációs backend végpont elkészítése során kialakítottam a felhasználók adatainak rögzítésére szolgáló API működését. A fejlesztés során a rendszer fogadja a frontendről érkező adatokat, majd ellenőrzi, hogy az adott email cím már szerepel-e az adatbázisban. Amennyiben az email cím még nem létezik, a rendszer eltárolja az új felhasználó adatait a users táblában, majd sikeres regisztrációs visszajelzést küld a kliens oldal számára.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    nev,
    email,
    jelszo,
    szerepkor,
    telefonszam
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    const letezik = await conn.query(
      "SELECT * FROM users WHERE email = ?",
      [email]
    );

    if (letezik.length > 0) {

      return res.json({
        siker: false,
        uzenet: "Ez az email mar letezik!"
      });

    }

    await conn.query(
      `INSERT INTO users
      (nev, email, jelszo, szerepkor, telefonszam)
      VALUES (?, ?, ?, ?, ?)`
    );

    [
      nev,
      email,
```

```

        jelszo,
        szerepkor,
        telefonszam
    ]
    );

    res.json({
        siker: true,
        uzenet: "Sikeres regisztracio!"
    });

    } catch (err) {

        res.status(500).json({
            siker: false,
            hiba: err
        });

    } finally {

        if (conn) conn.release();

    }

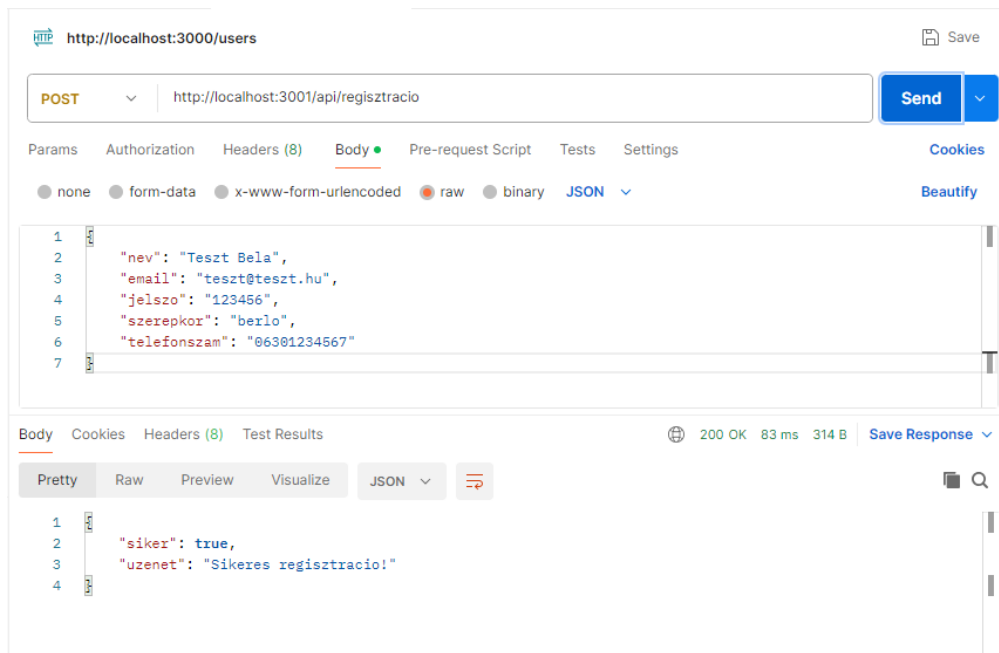
});

module.exports = router;

```

1.Részlet a regisztracio.js fájlból.

A regisztráció API tesztelése során Postman alkalmazás segítségével ellenőriztem a backend működését és az adatbázis kapcsolat helyességét. A teszt során JSON formátumban küldtem el a felhasználó adatait a regisztrációs végpontra, amelyet a rendszer sikeresen feldolgozott és eltárolt a MariaDB adatbázisban.



2.Részlet a regisztracio.js fájl Postman teszteléséből.

Bejelentkezés API elkészítése

A bejelentkezés API elkészítése során kialakítottam a felhasználók hitelesítéséhez szükséges backend végpontot. A fejlesztés során a rendszer ellenőrzi az adatbázisban tárolt email címet és jelszót, majd sikeres egyezés esetén visszaküldi a felhasználó legfontosabb adatait és a sikeres bejelentkezés állapotát. A működés tesztelésével ellenőriztem, hogy a backend megfelelően kezeli a felhasználói belépést és a MariaDB adatbázissal való kommunikációt.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    email,
    jelszo
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    const users = await conn.query(
      "SELECT * FROM users WHERE email = ? AND jelszo = ?",
      [email, jelszo]
    );

  }
```

```

    if (users.length === 0) {

        return res.json({
            siker: false,
            uzenet: "Hibas email vagy jelszo!"
        });

    }

    const user = users[0];

    res.json({
        siker: true,
        uzenet: "Sikeres bejelentkezés!",
        user: {
            id: user.id,
            nev: user.nev,
            email: user.email,
            szerepkor: user.szerepkor
        }
    });

} catch (err) {

    res.status(500).json({
        siker: false,
        hiba: err
    });

} finally {

    if (conn) conn.release();

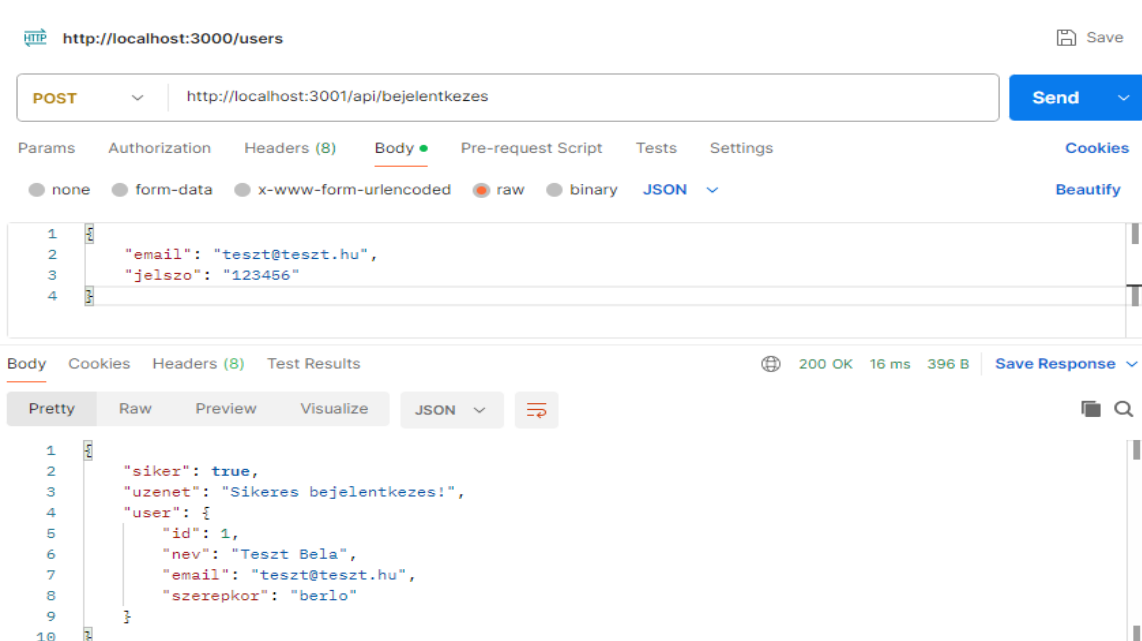
}

});

module.exports = router;

```

3.Részlet a bejelentkezés.js fájlból.



4. Részlet a `bejelentkezes.js` fájl Postman teszteléséből.

Users tábla létrehozása

A users tábla létrehozása során megterveztem és elkészítettem a rendszer felhasználóinak adatainak tárolására szolgáló adatbázis táblát. A tábla kialakításakor meghatároztam a szükséges mezőket, például a felhasználó nevét, email címét, jelszavát, szerepkörét és telefonszámát, valamint beállítottam az elsődleges kulcsot és az egyedi email azonosítást is. A létrehozás után SQL lekérdezéssel ellenőriztem a tábla sikeres működését és szerkezetét a MariaDB rendszerben.

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nev VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  jelszo VARCHAR(255) NOT NULL,
  szerepkor VARCHAR(50) NOT NULL,
  telefonszam VARCHAR(30),
  létrehozasdatum DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
MariaDB [ingatlankezeslo]> CREATE TABLE users (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   nev VARCHAR(100) NOT NULL,
  ->   email VARCHAR(100) NOT NULL UNIQUE,
  ->   jelszo VARCHAR(255) NOT NULL,
  ->   szerepkor VARCHAR(50) NOT NULL,
  ->   telefonszam VARCHAR(30),
  ->   létrehozasdatum DATETIME DEFAULT CURRENT_TIMESTAMP
  -> );
Query OK, 0 rows affected (0.027 sec)

MariaDB [ingatlankezeslo]> SELECT * FROM users;
Empty set (0.033 sec)
```

5. Részlet a `users` tábla létrehozásáról és az SQL kódjából.

Users tábla feltöltése

A users tábla feltöltése során létrehoztam a rendszer alapfelhasználóit, amelyek között adminisztrátorok és bérlők is szerepelnek. Az adatok rögzítése SQL INSERT utasítás segítségével történt, amely egyszerre több rekord beszúrását tette lehetővé. A felhasználókhoz nevet, e-mail címet, jelszót, szerepkört és telefonszámot adtam meg. A feltöltés után SQL lekérdezéssel ellenőriztem, hogy az adatok megfelelően bekerültek-e az adatbázisba. Az ellenőrzés során meggyőződtem arról, hogy a rendszerben tárolt felhasználók adatai helyesen jelennek meg.

```
INSERT INTO users (nev, email, jelszo, szerekor, telefonszam) VALUES
('Kovács István', 'istvan.kovacs@gmail.com', '1234', 'admin', '06301111111'),
('Nagy József', 'jozsef.nagy@gmail.com', '1234', 'admin', '06302222222'),
('Tóth Péter', 'peter.toth@gmail.com', '1234', 'berlo', '06303333333'),
('Kiss Anna', 'anna.kiss@gmail.com', '1234', 'berlo', '06304444444'),
('Szabó Dávid', 'david.szabo@gmail.com', '1234', 'berlo', '06305555555'),
('Varga Éva', 'eva.varga@gmail.com', '1234', 'berlo', '06306666666'),
('Molnár Gábor', 'gabor.molnar@gmail.com', '1234', 'berlo', '06307777777'),
('Farkas Dóra', 'dora.farkas@gmail.com', '1234', 'berlo', '06308888888'),
('Horváth Balázs', 'balazs.horvath@gmail.com', '1234', 'berlo', '06309999999'),
('Lakatos Krisztián', 'krisztian.lakatos@gmail.com', '1234', 'berlo', '06301234567'),
('Papp Renáta', 'renata.papp@gmail.com', '1234', 'berlo', '06307654321'),
('Bíró Zoltán', 'zoltan.biro@gmail.com', '1234', 'berlo', '06302345678');
```

```
MariaDB [ingatlankezo]> SELECT * FROM users;
```

id	nev	email	jelszo	szerekor	telefonszam	letrehoz
1	Teszt Bela	teszt@teszt.hu	123456	berlo	06301234567	2026-05-25 22:35:35
2	Kovács István	istvan.kovacs@gmail.com	1234	admin	06301111111	2026-06-02 22:30:14
3	Nagy József	jozsef.nagy@gmail.com	1234	admin	06302222222	2026-06-02 22:30:14
4	Tóth Péter	peter.toth@gmail.com	1234	berlo	06303333333	2026-06-02 22:30:14
5	Kiss Anna	anna.kiss@gmail.com	1234	berlo	06304444444	2026-06-02 22:30:14
6	Szabó Dávid	david.szabo@gmail.com	1234	berlo	06305555555	2026-06-02 22:30:14
7	Varga Éva	eva.varga@gmail.com	1234	berlo	06306666666	2026-06-02 22:30:14
8	Molnár Gábor	gabor.molnar@gmail.com	1234	berlo	06307777777	2026-06-02 22:30:14
9	Farkas Dóra	dora.farkas@gmail.com	1234	berlo	06308888888	2026-06-02 22:30:14
10	Horváth Balázs	balazs.horvath@gmail.com	1234	berlo	06309999999	2026-06-02 22:30:14
11	Lakatos Krisztián	krisztian.lakatos@gmail.com	1234	berlo	06301234567	2026-06-02 22:30:14
12	Papp Renáta	renata.papp@gmail.com	1234	berlo	06307654321	2026-06-02 22:30:14
13	Bíró Zoltán	zoltan.biro@gmail.com	1234	berlo	06302345678	2026-06-02 22:30:14

6.kép: A users adatbázis feltöltéséről.